```
pwd
```

```
'C:\\Users\\91831'
```

```
# ============================================================
# MACHINE LEARNING PROJECT
# WINE DATASET
# ============================================================

# INSTALL LIBRARIES
# pip install scikit-learn matplotlib seaborn pandas numpy ucimlrepo

# ============================================================
# IMPORT LIBRARIES
# ============================================================

import warnings
warnings.filterwarnings("ignore")

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from ucimlrepo import fetch_ucirepo

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import (
    StandardScaler,
    LabelEncoder
)

from sklearn.metrics import *

# ============================================================
# REGRESSION MODELS
# ============================================================

from sklearn.linear_model import (
    LinearRegression,
    LogisticRegression,
    Ridge,
    Lasso,
    Perceptron
)

# ============================================================
# CLASSIFICATION MODELS
# ============================================================
```

```python
from sklearn.neighbors import KNeighborsClassifier

from sklearn.tree import (
    DecisionTreeClassifier,
    plot_tree
)

from sklearn.naive_bayes import GaussianNB

from sklearn.svm import SVC

# ============================================================
# ENSEMBLE MODELS
# ============================================================

from sklearn.ensemble import (
    RandomForestClassifier,
    AdaBoostClassifier,
    GradientBoostingClassifier,
    ExtraTreesClassifier
)

# ============================================================
# NEURAL NETWORK
# ============================================================

from sklearn.neural_network import MLPClassifier

# ============================================================
# PCA
# ============================================================

from sklearn.decomposition import PCA

# ============================================================
# CLUSTERING
# ============================================================

from sklearn.cluster import (
    KMeans,
    DBSCAN,
    AgglomerativeClustering
)

# ============================================================
# LOAD DATASET
# WINE DATASET
# ============================================================

dataset = fetch_ucirepo(id=109)
```

```python
X = dataset.data.features.copy()

y = dataset.data.targets.squeeze().copy()

# Encode Target

y = LabelEncoder().fit_transform(y)

print("Dataset Shape:", X.shape)

print("\nFirst 5 Rows:\n")

print(X.head())

# ============================================================
# MISSING VALUES
# ============================================================

print("\nMissing Values:\n")

print(X.isnull().sum())

# ============================================================
# DATASET INFO
# ============================================================

print("\nDataset Information:\n")

print(X.info())

# ============================================================
# HEATMAP
# ============================================================

plt.figure(figsize=(12,8))

sns.heatmap(
    X.corr(),
    annot=True,
    cmap="coolwarm"
)

plt.title("Correlation Heatmap")

plt.show()

# ============================================================
# OUTLIER DETECTION
# ============================================================

plt.figure(figsize=(14,6))
```

```
sns.boxplot(data=X)

plt.xticks(rotation=90)

plt.title("Outlier Detection")

plt.show()

# ============================================================
# DISTRIBUTION PLOTS
# ============================================================

X.hist(
    figsize=(14,10),
    bins=20
)

plt.suptitle("Feature Distribution")

plt.show()

# ============================================================
# FEATURE SCALING
# ============================================================

scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)

# ============================================================
# PCA
# ============================================================

pca = PCA(n_components=2)

X_pca = pca.fit_transform(X_scaled)

plt.figure(figsize=(8,6))

plt.scatter(
    X_pca[:,0],
    X_pca[:,1],
    c=y
)

plt.title("PCA Visualization")

plt.xlabel("PC1")

plt.ylabel("PC2")
```

```python
plt.grid()

plt.show()

# ============================================================
# TRAIN TEST SPLIT
# ============================================================

X_train, X_test, y_train, y_test = train_test_split(
    X_scaled,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

# ============================================================
# ALL MODELS
# ============================================================

models = {

    "Linear Regression":
    LinearRegression(),

    "Logistic Regression":
    LogisticRegression(max_iter=5000),

    "Ridge Regression":
    Ridge(alpha=1),

    "Lasso Regression":
    Lasso(alpha=0.01),

    "KNN":
    KNeighborsClassifier(n_neighbors=5),

    "Decision Tree":
    DecisionTreeClassifier(
        max_depth=5,
        random_state=42
    ),

    "Naive Bayes":
    GaussianNB(),

    "SVM Linear":
    SVC(kernel="linear"),

    "SVM RBF":
```

```
    SVC(kernel="rbf"),

    "Random Forest":
    RandomForestClassifier(
        n_estimators=200,
        random_state=42
    ),

    "AdaBoost":
    AdaBoostClassifier(
        n_estimators=100,
        random_state=42
    ),

    "Gradient Boosting":
    GradientBoostingClassifier(),

    "Extra Trees":
    ExtraTreesClassifier(
        n_estimators=200,
        random_state=42
    ),

    "Neural Network":
    MLPClassifier(
        hidden_layer_sizes=(64,32),
        max_iter=2000,
        random_state=42
    ),

    "Perceptron":
    Perceptron()
}

# ============================================================
# STORAGE
# ============================================================

model_names = []

model_scores = []

# ============================================================
# TRAIN MODELS
# ============================================================

for name, model in models.items():

    print("\n==================================================")
```

```
    print(name)

    print("==================================================")

    # Train

    model.fit(X_train, y_train)

    # Prediction

    pred = model.predict(X_test)

    # Regression Adjustment

    if name in [

        "Linear Regression",
        "Ridge Regression",
        "Lasso Regression"

    ]:

        pred = np.round(pred).astype(int)

        pred = np.clip(
            pred,
            y.min(),
            y.max()
        )

    # Accuracy

    accuracy = accuracy_score(
        y_test,
        pred
    )

    model_names.append(name)

    model_scores.append(accuracy)

    print("\nAccuracy:", accuracy)

    # Classification Report

    print("\nClassification Report:\n")

    print(
        classification_report(
            y_test,
            pred
```

```
        )
    )

    # ========================================================
    # CONFUSION MATRIX
    # ========================================================

    cm = confusion_matrix(
        y_test,
        pred
    )

    plt.figure(figsize=(6,4))

    sns.heatmap(
        cm,
        annot=True,
        fmt="d",
        cmap="Blues"
    )

    plt.title(f"{name} Confusion Matrix")

    plt.xlabel("Predicted")

    plt.ylabel("Actual")

    plt.show()

    # ========================================================
    # DECISION TREE VISUALIZATION
    # ========================================================

    if name == "Decision Tree":

        plt.figure(figsize=(18,8))

        plot_tree(
            model,
            filled=True,
            fontsize=8
        )

        plt.title("Decision Tree")

        plt.show()

    # ========================================================
    # FEATURE IMPORTANCE
    # ========================================================
```

```
    if name in [

        "Random Forest",
        "Extra Trees",
        "Gradient Boosting"

    ]:

        importance = model.feature_importances_

        indices = np.argsort(
            importance
        )[::-1]

        plt.figure(figsize=(10,5))

        plt.bar(
            range(X.shape[1]),
            importance[indices]
        )

        plt.xticks(
            range(X.shape[1]),
            X.columns[indices],
            rotation=90
        )

        plt.title(
            f"{name} Feature Importance"
        )

        plt.show()

    # ======================================================
    # NEURAL NETWORK LOSS
    # ======================================================

    if name == "Neural Network":

        plt.figure(figsize=(6,4))

        plt.plot(
            model.loss_curve_
        )

        plt.title(
            "Neural Network Loss Curve"
        )

        plt.xlabel("Iterations")
```

```
        plt.ylabel("Loss")

        plt.grid()

        plt.show()

# ============================================================
# K-MEANS
# ============================================================

kmeans = KMeans(
    n_clusters=3,
    random_state=42
)

kmeans_labels = kmeans.fit_predict(X_scaled)

plt.figure(figsize=(8,6))

plt.scatter(
    X_pca[:,0],
    X_pca[:,1],
    c=kmeans_labels
)

plt.title("K-Means Clustering")

plt.grid()

plt.show()

# ============================================================
# ELBOW METHOD
# ============================================================

wcss = []

for i in range(1,11):

    km = KMeans(
        n_clusters=i,
        random_state=42
    )

    km.fit(X_scaled)

    wcss.append(km.inertia_)

plt.figure(figsize=(8,5))

plt.plot(
```

```
    range(1,11),
    wcss,
    marker='o'
)

plt.title("Elbow Method")

plt.xlabel("Clusters")

plt.ylabel("WCSS")

plt.grid()

plt.show()

# ============================================================
# DBSCAN
# ============================================================

dbscan = DBSCAN(
    eps=2,
    min_samples=5
)

dbscan_labels = dbscan.fit_predict(X_scaled)

plt.figure(figsize=(8,6))

plt.scatter(
    X_pca[:,0],
    X_pca[:,1],
    c=dbscan_labels
)

plt.title("DBSCAN Clustering")

plt.grid()

plt.show()

# ============================================================
# HIERARCHICAL CLUSTERING
# ============================================================

hc = AgglomerativeClustering(
    n_clusters=3
)

hc_labels = hc.fit_predict(X_scaled)

plt.figure(figsize=(8,6))
```

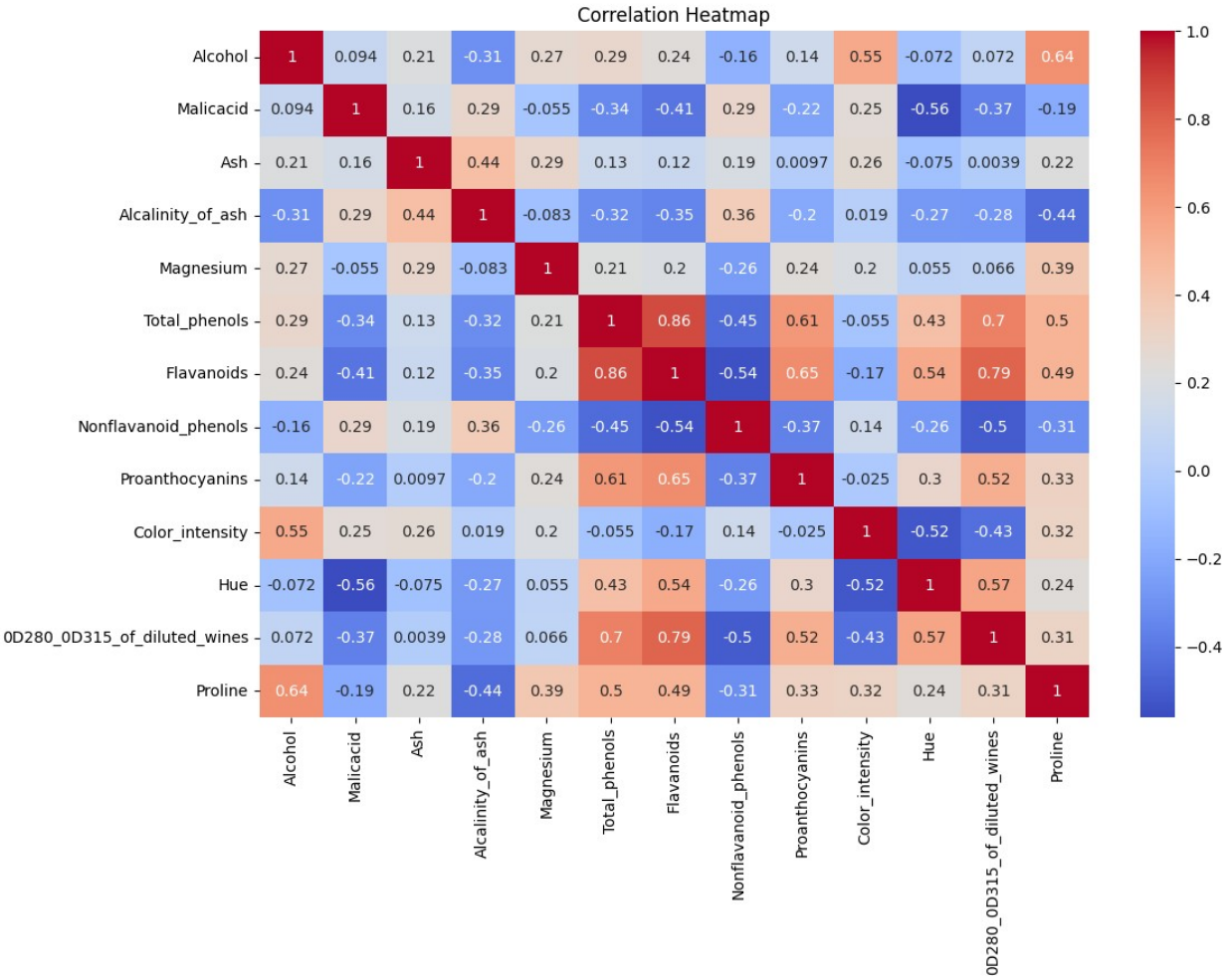
```
plt.scatter(
    X_pca[:,0],
    X_pca[:,1],
    c=hc_labels
)

plt.title("Hierarchical Clustering")

plt.grid()

plt.show()

# ============================================================
# MODEL COMPARISON BAR GRAPH
# ============================================================

plt.figure(figsize=(16,6))

bars = plt.bar(
    model_names,
    model_scores
)

plt.title("Model Accuracy Comparison")

plt.ylabel("Accuracy")

plt.xticks(rotation=45)

for bar in bars:

    yval = bar.get_height()

    plt.text(
        bar.get_x() + 0.1,
        yval + 0.01,
        round(yval, 2)
    )

plt.grid(axis="y")

plt.show()

# ============================================================
# MODEL COMPARISON LINE GRAPH
# ============================================================

plt.figure(figsize=(16,6))

plt.plot(
```

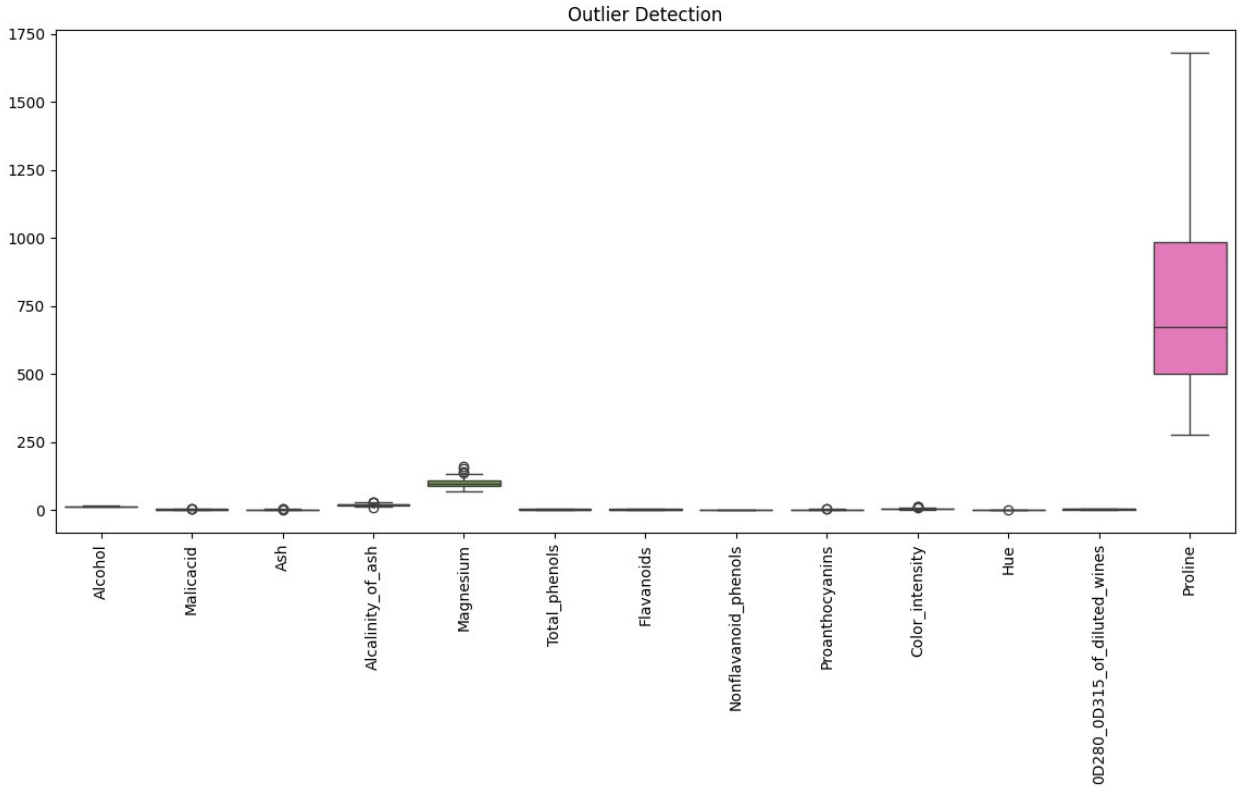
```
    model_names,
    model_scores,
    marker='o'
)

plt.title("Model Accuracy Line Plot")

plt.ylabel("Accuracy")

plt.xticks(rotation=45)

plt.grid()

plt.show()

# ==========================================================
# PIE CHART
# ==========================================================

plt.figure(figsize=(10,10))

plt.pie(
    model_scores,
    labels=model_names,
    autopct='%1.1f%%'
)

plt.title("Model Accuracy Share")

plt.show()

# ==========================================================
# CLASS DISTRIBUTION
# ==========================================================

plt.figure(figsize=(6,4))

sns.countplot(x=y)

plt.title("Class Distribution")

plt.show()

# ==========================================================
# BEST MODEL
# ==========================================================

best_index = np.argmax(model_scores)

print("\n==================================================")
```

```python
print("BEST MODEL:", model_names[best_index])

print("BEST ACCURACY:", model_scores[best_index])

print("==================================================")

# ==========================================================
# END
# ==========================================================
```

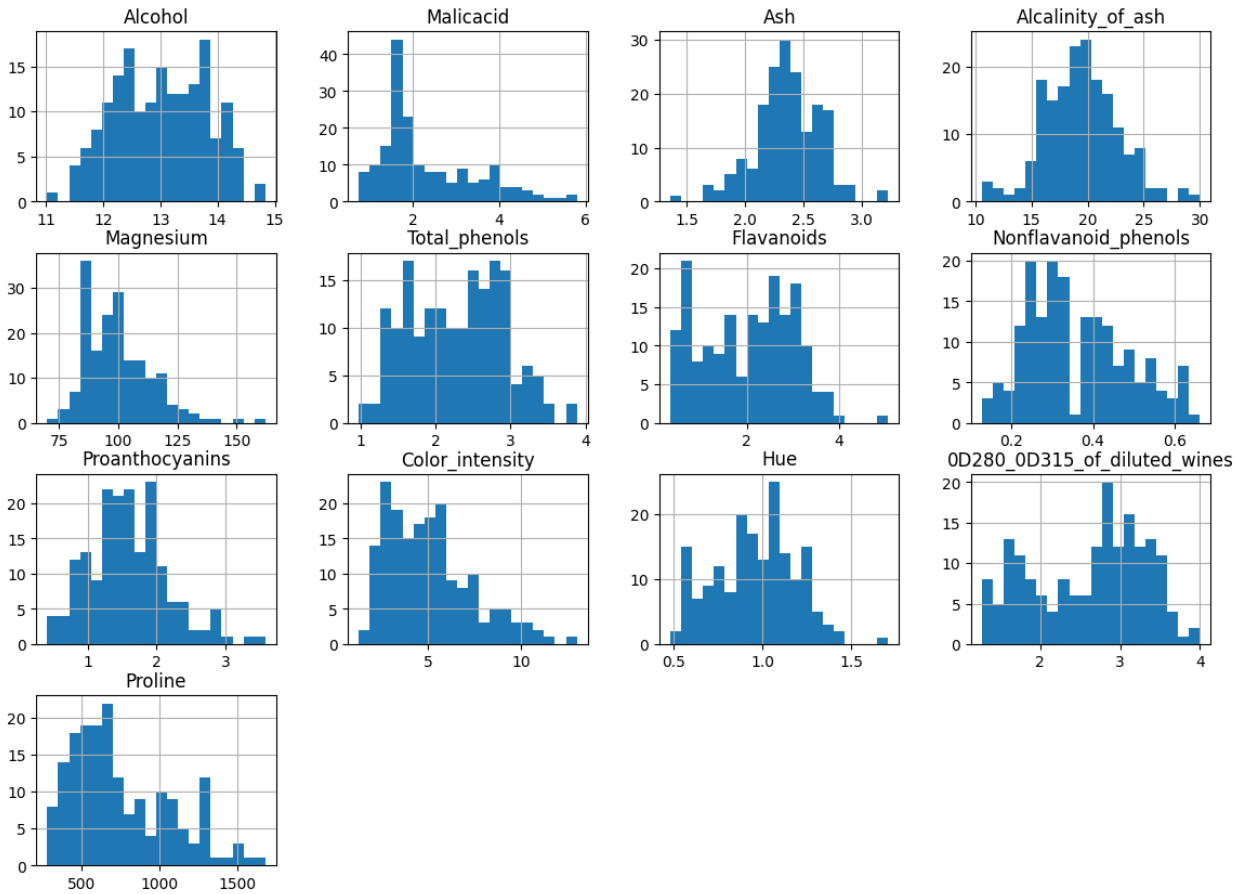
```
Dataset Shape: (178, 13)

First 5 Rows:

   Alcohol  Malicacid   Ash  Alcalinity_of_ash  Magnesium
Total_phenols  \
0    14.23       1.71  2.43               15.6        127
2.80
1    13.20       1.78  2.14               11.2        100
2.65
2    13.16       2.36  2.67               18.6        101
2.80
3    14.37       1.95  2.50               16.8        113
3.85
4    13.24       2.59  2.87               21.0        118
2.80

   Flavanoids  Nonflavanoid_phenols  Proanthocyanins  Color_intensity
Hue  \
0        3.06                  0.28             2.29             5.64
1.04
1        2.76                  0.26             1.28             4.38
1.05
2        3.24                  0.30             2.81             5.68
1.03
3        3.49                  0.24             2.18             7.80
0.86
4        2.69                  0.39             1.82             4.32
1.04

   0D280_0D315_of_diluted_wines  Proline
0                          3.92     1065
1                          3.40     1050
2                          3.17     1185
3                          3.45     1480
4                          2.93      735

Missing Values:

Alcohol                         0
```

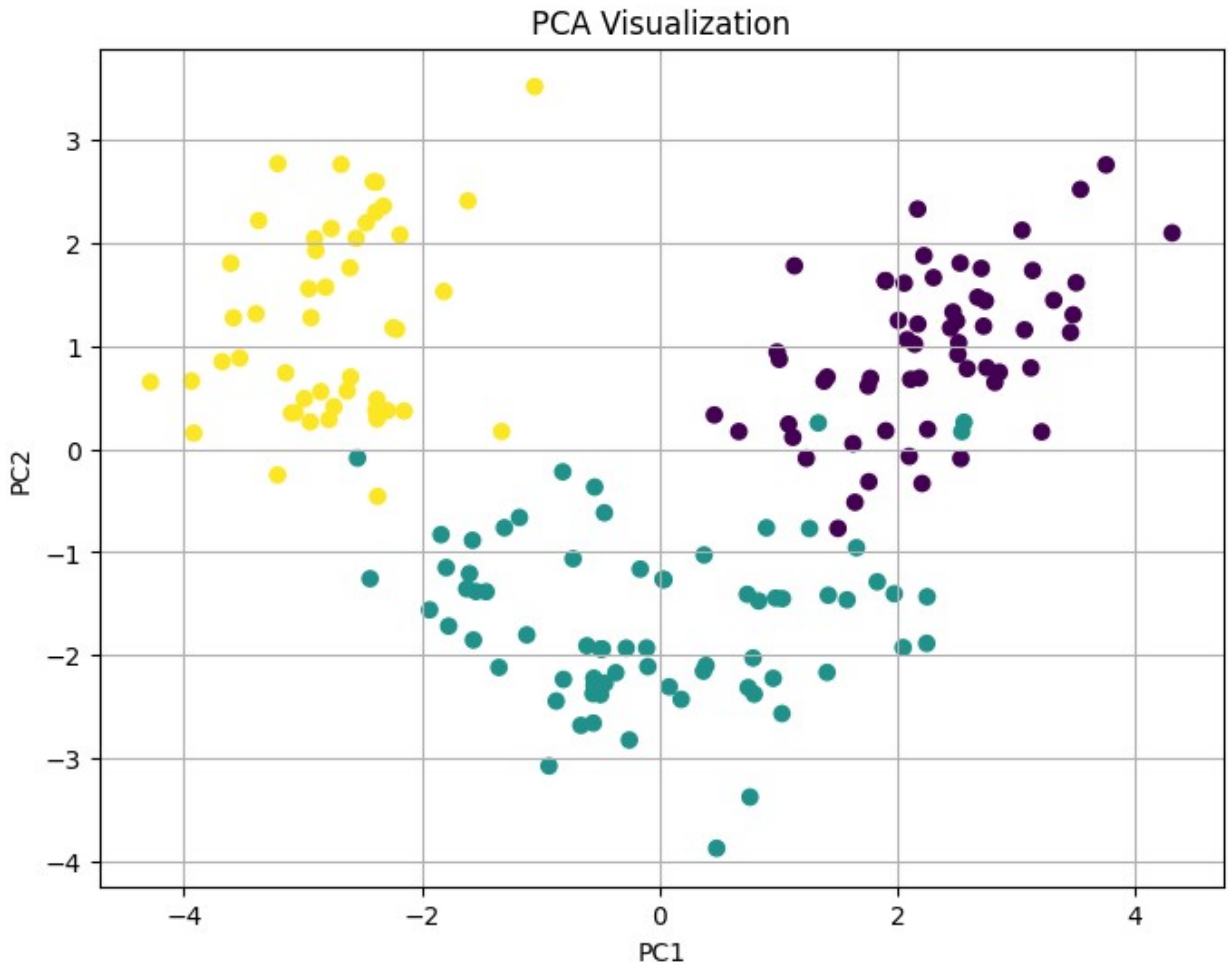
```
Malicacid                         0
Ash                               0
Alcalinity_of_ash                 0
Magnesium                         0
Total_phenols                     0
Flavanoids                        0
Nonflavanoid_phenols              0
Proanthocyanins                   0
Color_intensity                   0
Hue                               0
0D280_0D315_of_diluted_wines      0
Proline                           0
dtype: int64

Dataset Information:

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 178 entries, 0 to 177
Data columns (total 13 columns):
 #   Column                        Non-Null Count  Dtype
---  ------                        --------------  -----
 0   Alcohol                       178 non-null    float64
 1   Malicacid                     178 non-null    float64
 2   Ash                           178 non-null    float64
 3   Alcalinity_of_ash             178 non-null    float64
 4   Magnesium                     178 non-null    int64
 5   Total_phenols                 178 non-null    float64
 6   Flavanoids                    178 non-null    float64
 7   Nonflavanoid_phenols          178 non-null    float64
 8   Proanthocyanins               178 non-null    float64
 9   Color_intensity               178 non-null    float64
 10  Hue                           178 non-null    float64
 11  0D280_0D315_of_diluted_wines  178 non-null    float64
 12  Proline                       178 non-null    int64
dtypes: float64(11), int64(2)
memory usage: 18.2 KB
None
```


Correlation Heatmap
Alcohol 1 0.094 0.21 -0.31 0.27 0.29 0.24 -0.16 0.14 0.55 -0.072 0.072 0.64
Malicacid 0.094 1 0.16 0.29 -0.055 -0.34 -0.41 0.29 -0.22 0.25 -0.56 -0.37 -0.19
Ash 0.21 0.16 1 0.44 0.29 0.13 0.12 0.19 0.0097 0.26 -0.075 0.0039 0.22
Alcalinity_of_ash -0.31 0.29 0.44 1 -0.083 -0.32 -0.35 0.36 -0.2 0.019 -0.27 -0.28 -0.44
Magnesium 0.27 -0.055 0.29 -0.083 1 0.21 0.2 -0.26 0.24 0.2 0.055 0.066 0.39
Total_phenols 0.29 -0.34 0.13 -0.32 0.21 1 0.86 -0.45 0.61 -0.055 0.43 0.7 0.5
Flavanoids 0.24 -0.41 0.12 -0.35 0.2 0.86 1 -0.54 0.65 -0.17 0.54 0.79 0.49
Nonflavanoid_phenols -0.16 0.29 0.19 0.36 -0.26 -0.45 -0.54 1 -0.37 0.14 -0.26 -0.5 -0.31
Proanthocyanins 0.14 -0.22 0.0097 -0.2 0.24 0.61 0.65 -0.37 1 -0.025 0.3 0.52 0.33
Color_intensity 0.55 0.25 0.26 0.019 0.2 -0.055 -0.17 0.14 -0.025 1 -0.52 -0.43 0.32
Hue -0.072 -0.56 -0.075 -0.27 0.055 0.43 0.54 -0.26 0.3 -0.52 1 0.57 0.24
0D280_0D315_of_diluted_wines 0.072 -0.37 0.0039 -0.28 0.066 0.7 0.79 -0.5 0.52 -0.43 0.57 1 0.31
Proline 0.64 -0.19 0.22 -0.44 0.39 0.5 0.49 -0.31 0.33 0.32 0.24 0.31 1
Alcohol
Malicacid
Ash
Alcalinity_of_ash
Magnesium
Total_phenols
Flavanoids
Nonflavanoid_phenols
Proanthocyanins
Color_intensity
Hue
0D280_0D315_of_diluted_wines
Proline
1.0
0.8
0.6
0.4
0.2
0.0
−0.2
−0.4

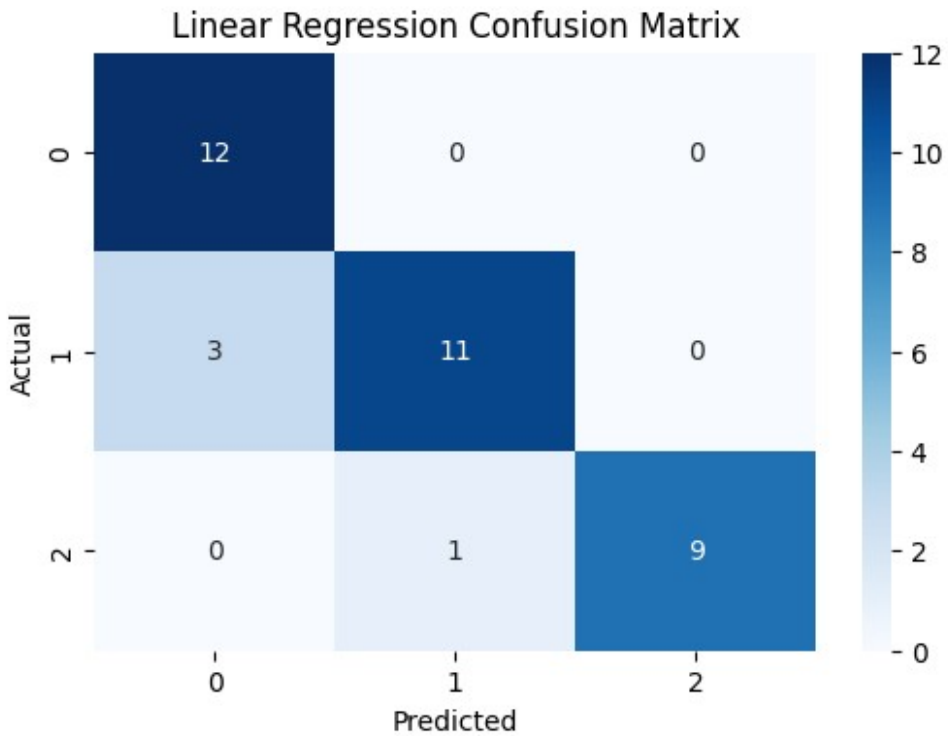

Outlier Detection
1750
1500
1250
1000
750
500
250
0
Alcohol
Malicacid
Ash
Alcalinity_of_ash
Magnesium
Total_phenols
Flavanoids
Nonflavanoid_phenols
Proanthocyanins
Color_intensity
Hue
OD280_OD315_of_diluted_wines
Proline

Feature Distribution

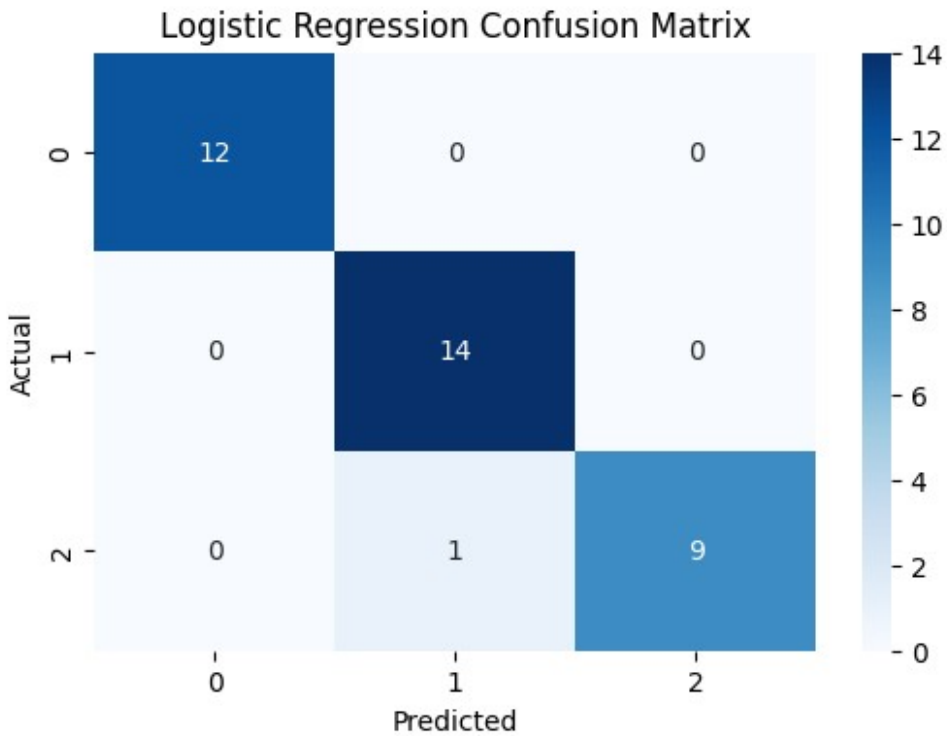




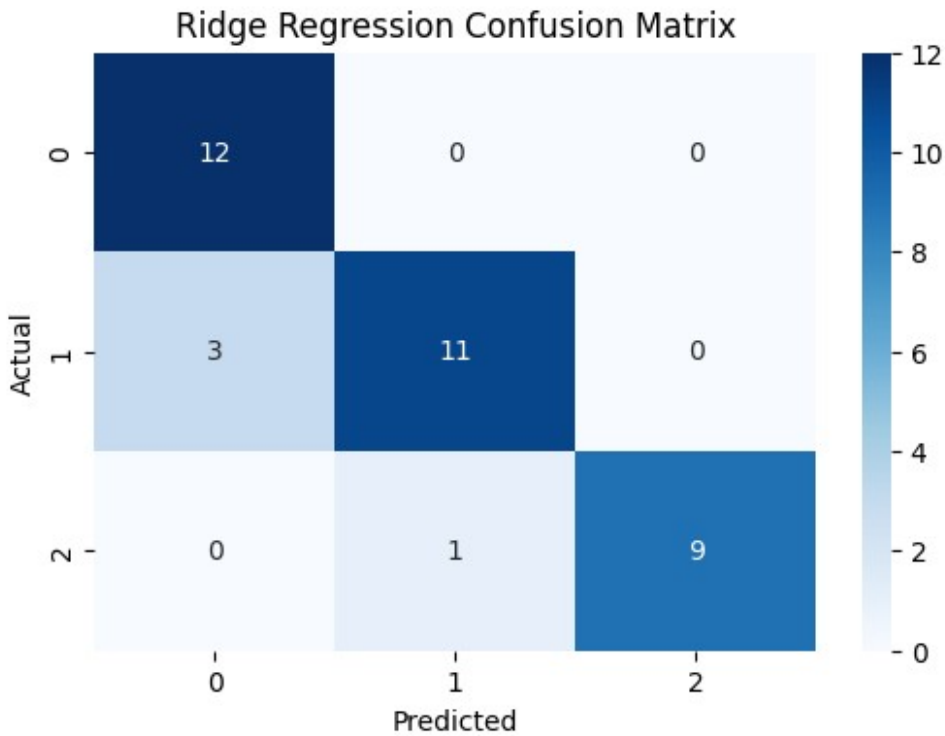

```
==================================================
Linear Regression
==================================================

Accuracy: 0.8888888888888888

Classification Report:

              precision    recall  f1-score   support

           0       0.80      1.00      0.89        12
           1       0.92      0.79      0.85        14
           2       1.00      0.90      0.95        10

    accuracy                           0.89        36
   macro avg       0.91      0.90      0.89        36
weighted avg       0.90      0.89      0.89        36
```



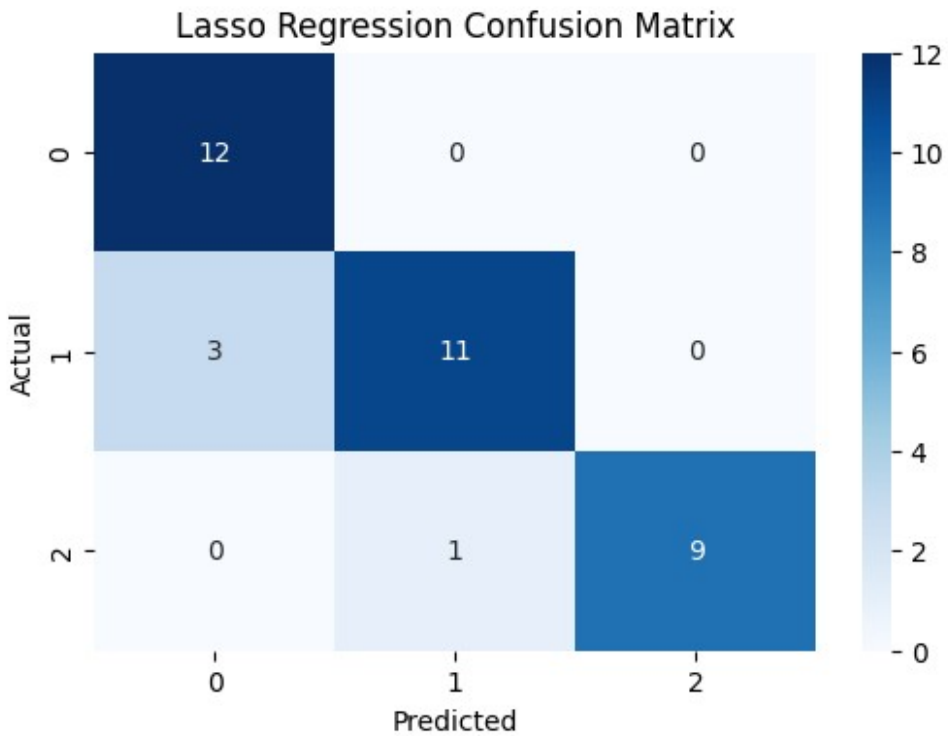

```
==================================================
Logistic Regression
==================================================

Accuracy: 0.9722222222222222

Classification Report:

              precision    recall  f1-score   support

           0       1.00      1.00      1.00        12
           1       0.93      1.00      0.97        14
           2       1.00      0.90      0.95        10

    accuracy                           0.97        36
   macro avg       0.98      0.97      0.97        36
weighted avg       0.97      0.97      0.97        36
```



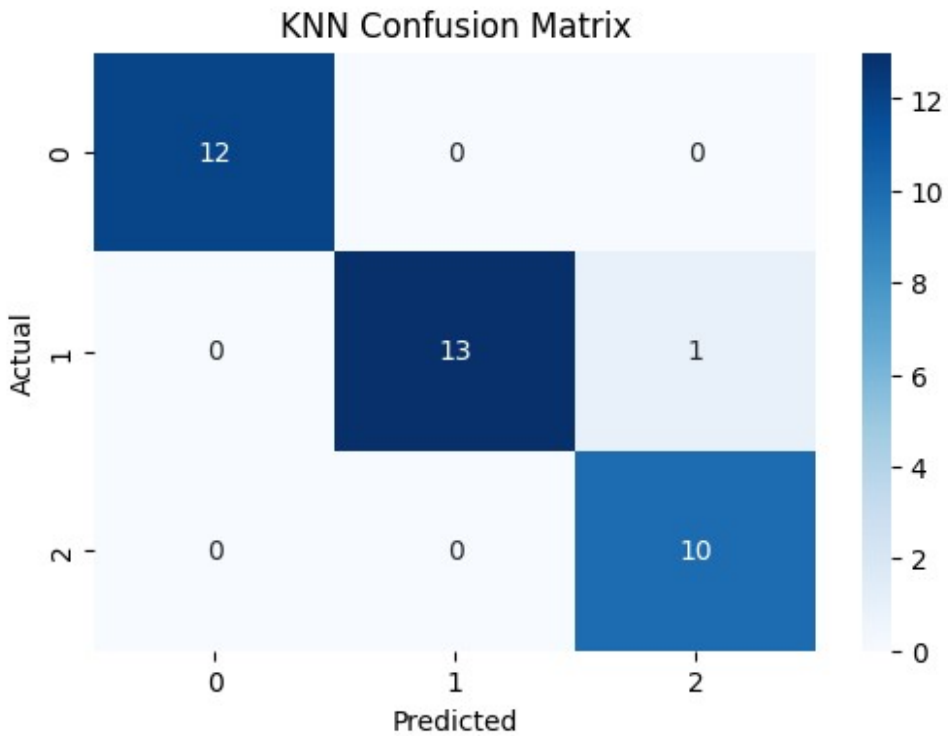

```
==================================================
Ridge Regression
==================================================

Accuracy: 0.8888888888888888

Classification Report:

              precision    recall  f1-score   support

           0       0.80      1.00      0.89        12
           1       0.92      0.79      0.85        14
           2       1.00      0.90      0.95        10

    accuracy                           0.89        36
   macro avg       0.91      0.90      0.89        36
weighted avg       0.90      0.89      0.89        36
```



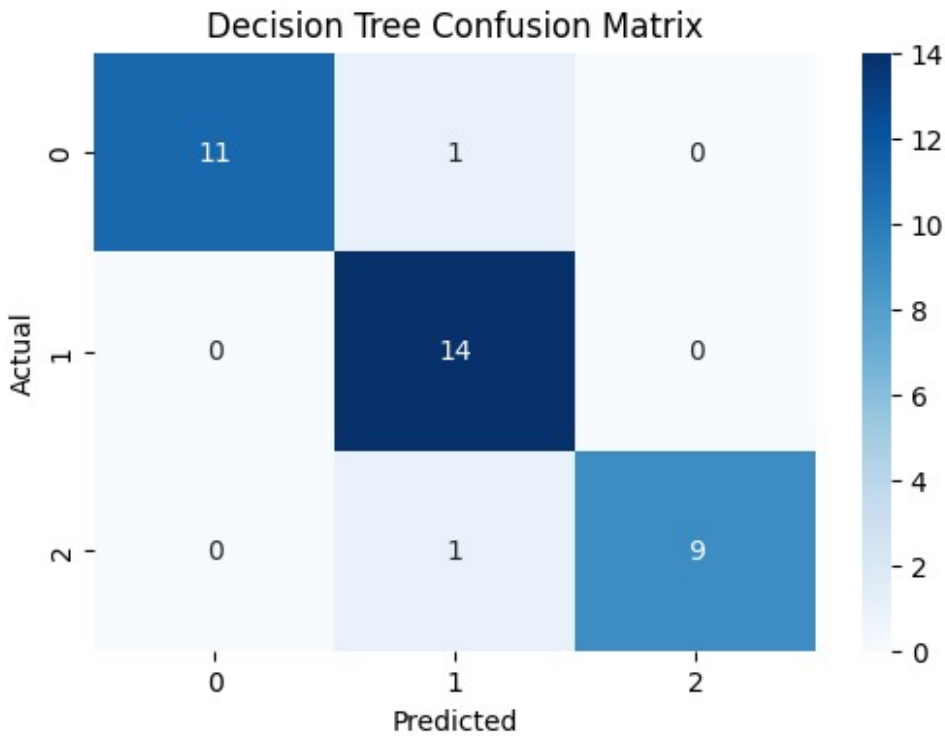

```
==================================================
Lasso Regression
==================================================

Accuracy: 0.8888888888888888

Classification Report:

              precision    recall  f1-score   support

           0       0.80      1.00      0.89        12
           1       0.92      0.79      0.85        14
           2       1.00      0.90      0.95        10

    accuracy                           0.89        36
   macro avg       0.91      0.90      0.89        36
weighted avg       0.90      0.89      0.89        36
```

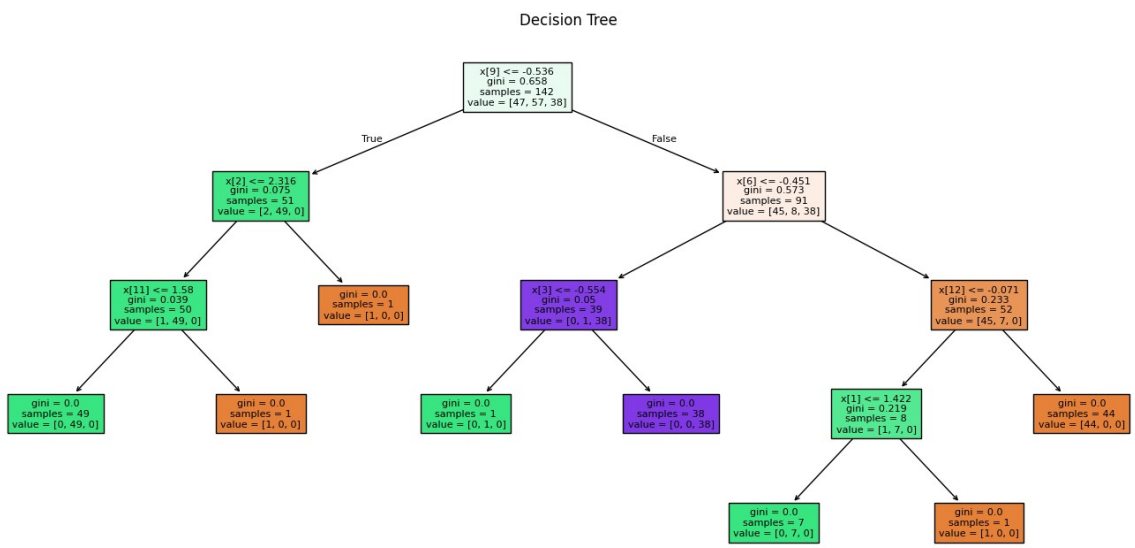



```
==================================================
KNN
==================================================

Accuracy: 0.9722222222222222

Classification Report:

              precision    recall  f1-score   support

           0       1.00      1.00      1.00        12
           1       1.00      0.93      0.96        14
           2       0.91      1.00      0.95        10

    accuracy                           0.97        36
   macro avg       0.97      0.98      0.97        36
weighted avg       0.97      0.97      0.97        36
```



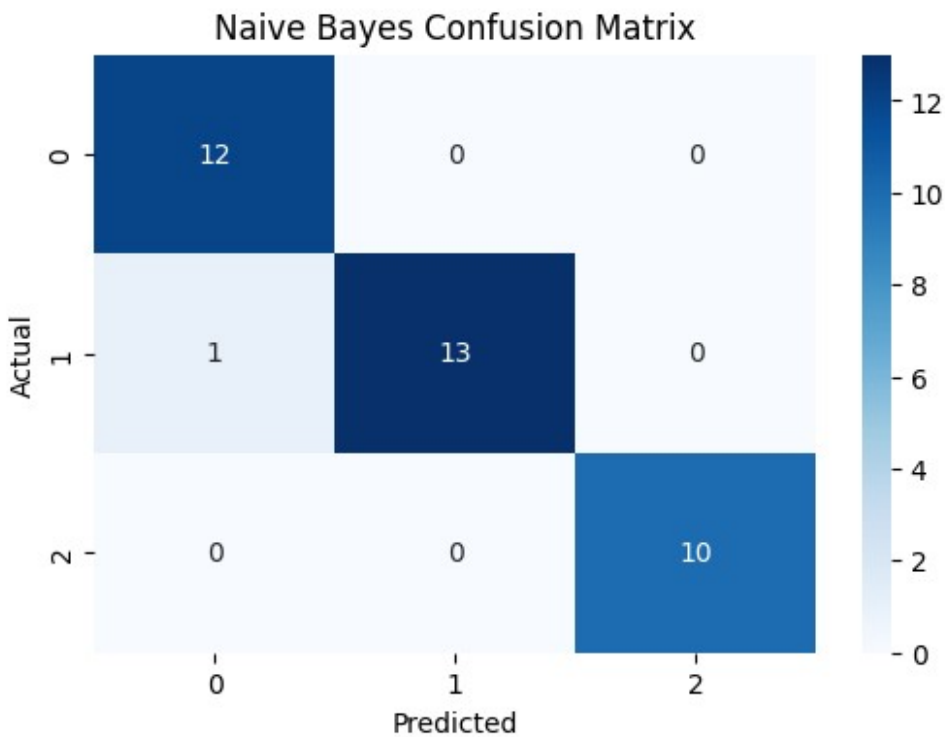

```
==================================================
Decision Tree
==================================================

Accuracy: 0.9444444444444444

Classification Report:

              precision    recall  f1-score   support

           0       1.00      0.92      0.96        12
           1       0.88      1.00      0.93        14
           2       1.00      0.90      0.95        10

    accuracy                           0.94        36
   macro avg       0.96      0.94      0.95        36
weighted avg       0.95      0.94      0.94        36
```





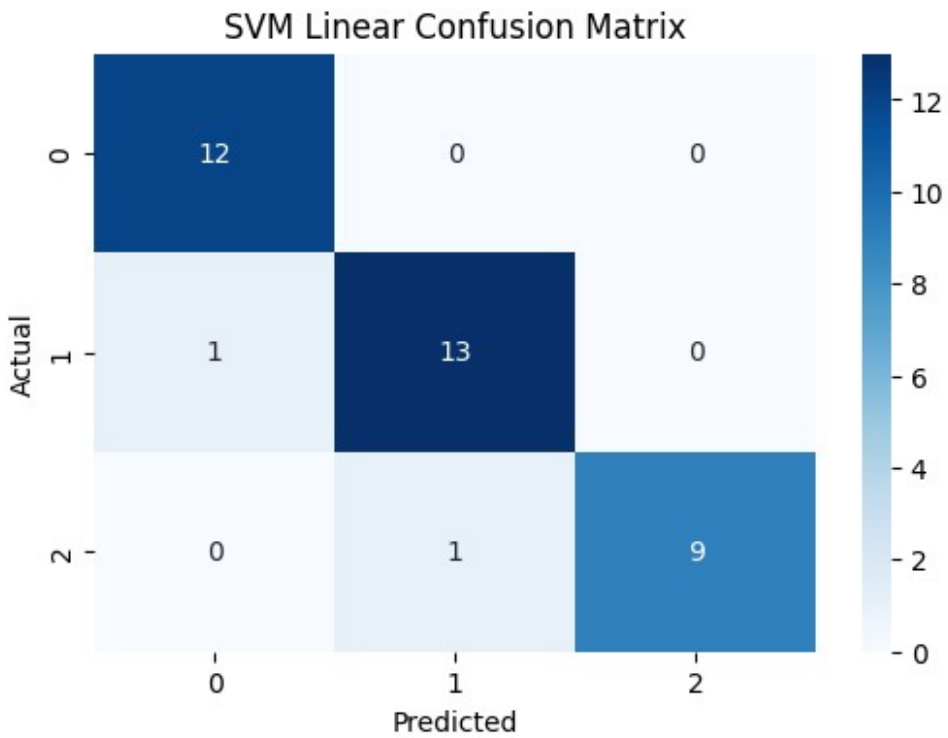

```
==================================================
Naive Bayes
==================================================

Accuracy: 0.9722222222222222

Classification Report:
```

```
              precision    recall  f1-score   support

           0       0.92      1.00      0.96        12
           1       1.00      0.93      0.96        14
           2       1.00      1.00      1.00        10

    accuracy                           0.97        36
   macro avg       0.97      0.98      0.97        36
weighted avg       0.97      0.97      0.97        36
```

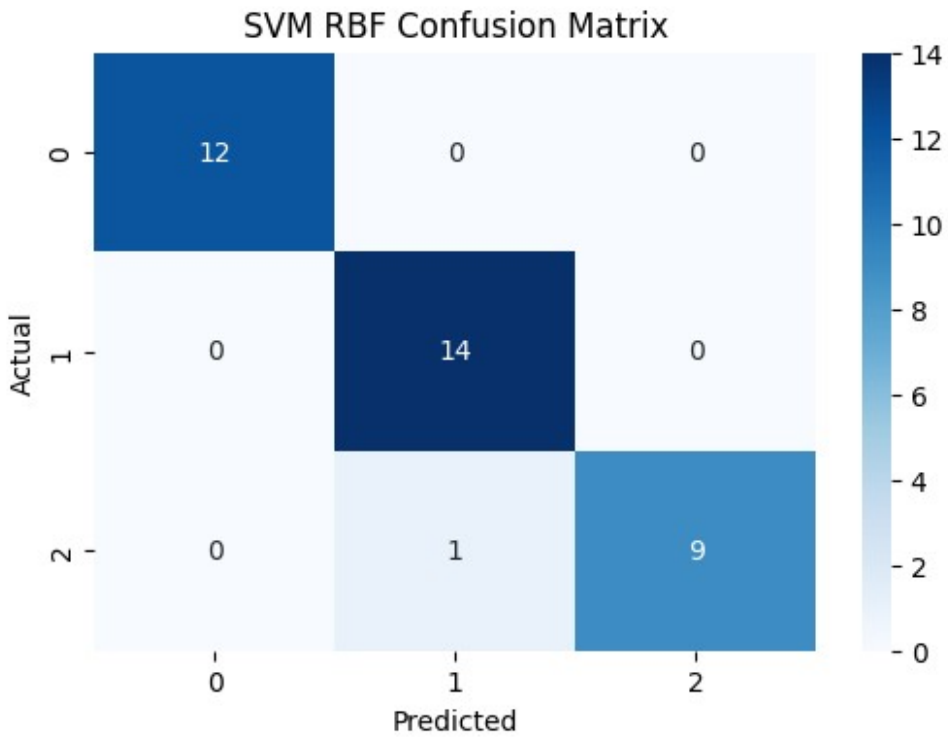



```
==================================================
SVM Linear
==================================================

Accuracy: 0.9444444444444444

Classification Report:

              precision    recall  f1-score   support

           0       0.92      1.00      0.96        12
           1       0.93      0.93      0.93        14
           2       1.00      0.90      0.95        10
```

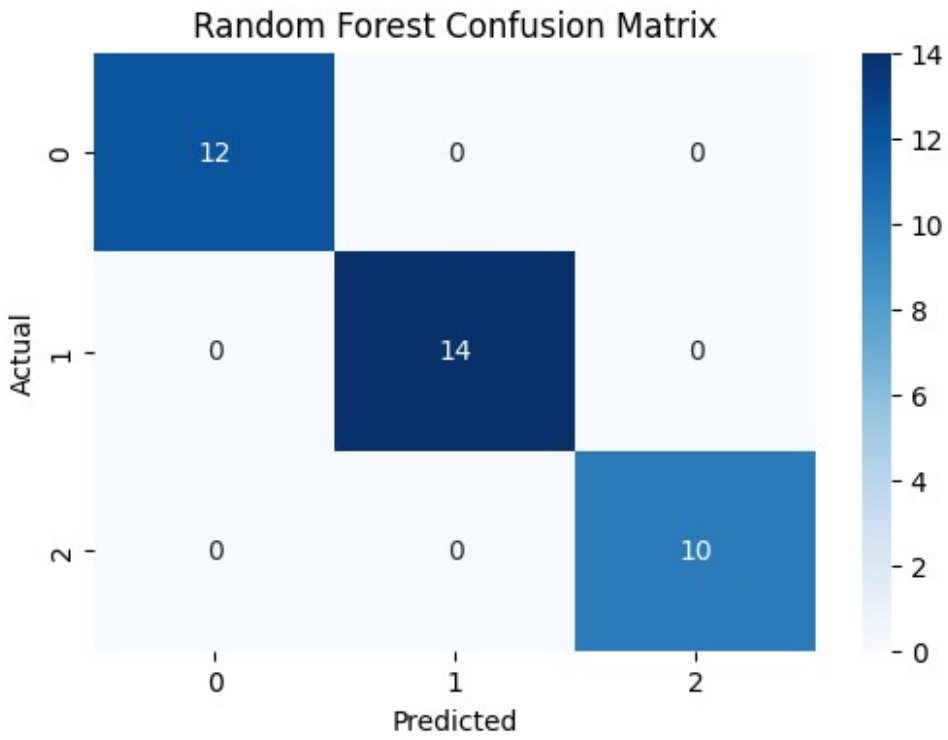
```
    accuracy                           0.94        36
   macro avg       0.95      0.94      0.95        36
weighted avg       0.95      0.94      0.94        36
```

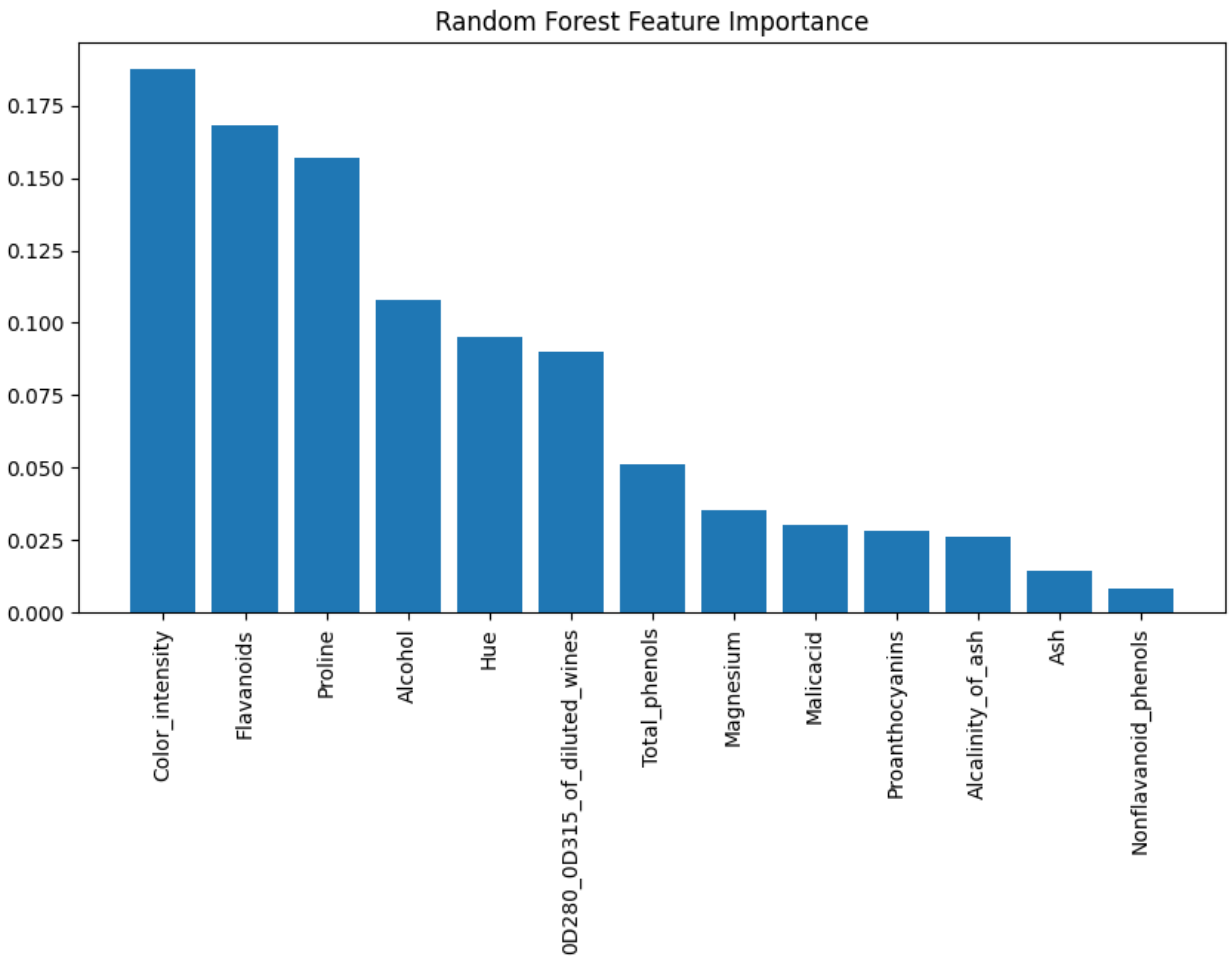



```
==================================================
SVM RBF
==================================================

Accuracy: 0.9722222222222222

Classification Report:

              precision    recall  f1-score   support

           0       1.00      1.00      1.00        12
           1       0.93      1.00      0.97        14
           2       1.00      0.90      0.95        10

    accuracy                           0.97        36
   macro avg       0.98      0.97      0.97        36
weighted avg       0.97      0.97      0.97        36
```

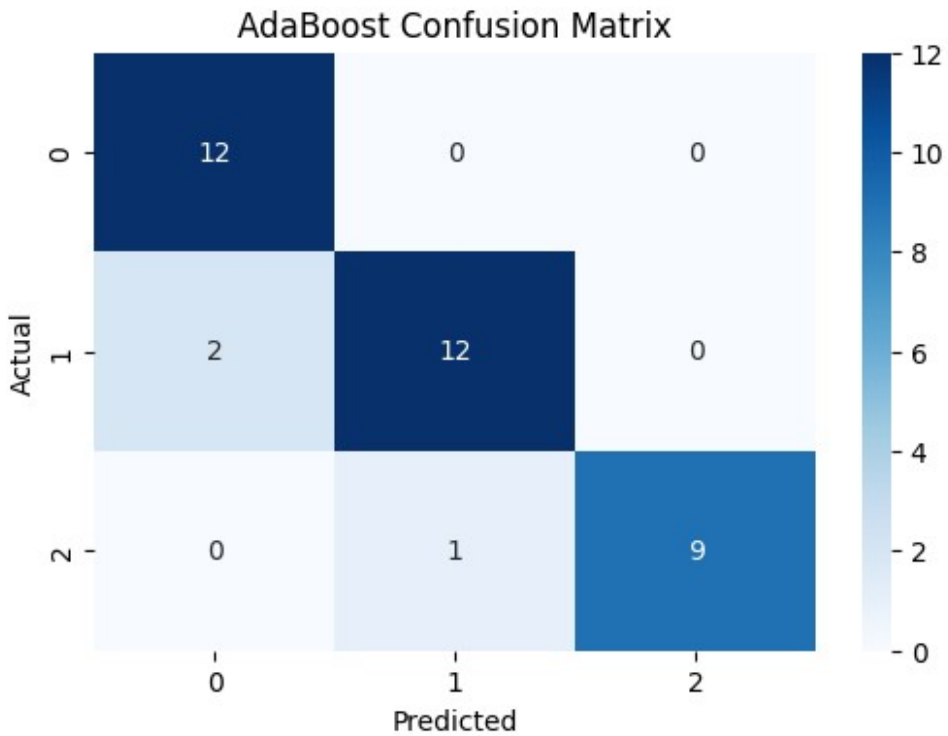



```
==================================================
Random Forest
==================================================

Accuracy: 1.0

Classification Report:

              precision    recall  f1-score   support

           0       1.00      1.00      1.00        12
           1       1.00      1.00      1.00        14
           2       1.00      1.00      1.00        10

    accuracy                           1.00        36
   macro avg       1.00      1.00      1.00        36
weighted avg       1.00      1.00      1.00        36
```

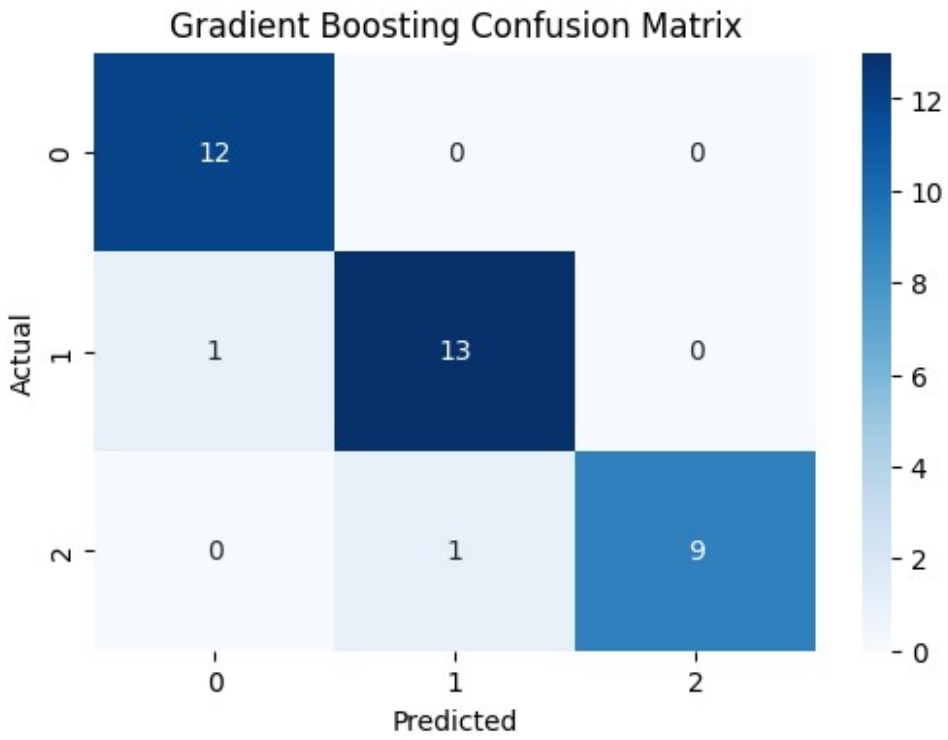

Random Forest Confusion Matrix
Actual
Predicted
0
1
2
12
0
0
0
14
0
0
0
10
14
12
10
8
6
4
2
0

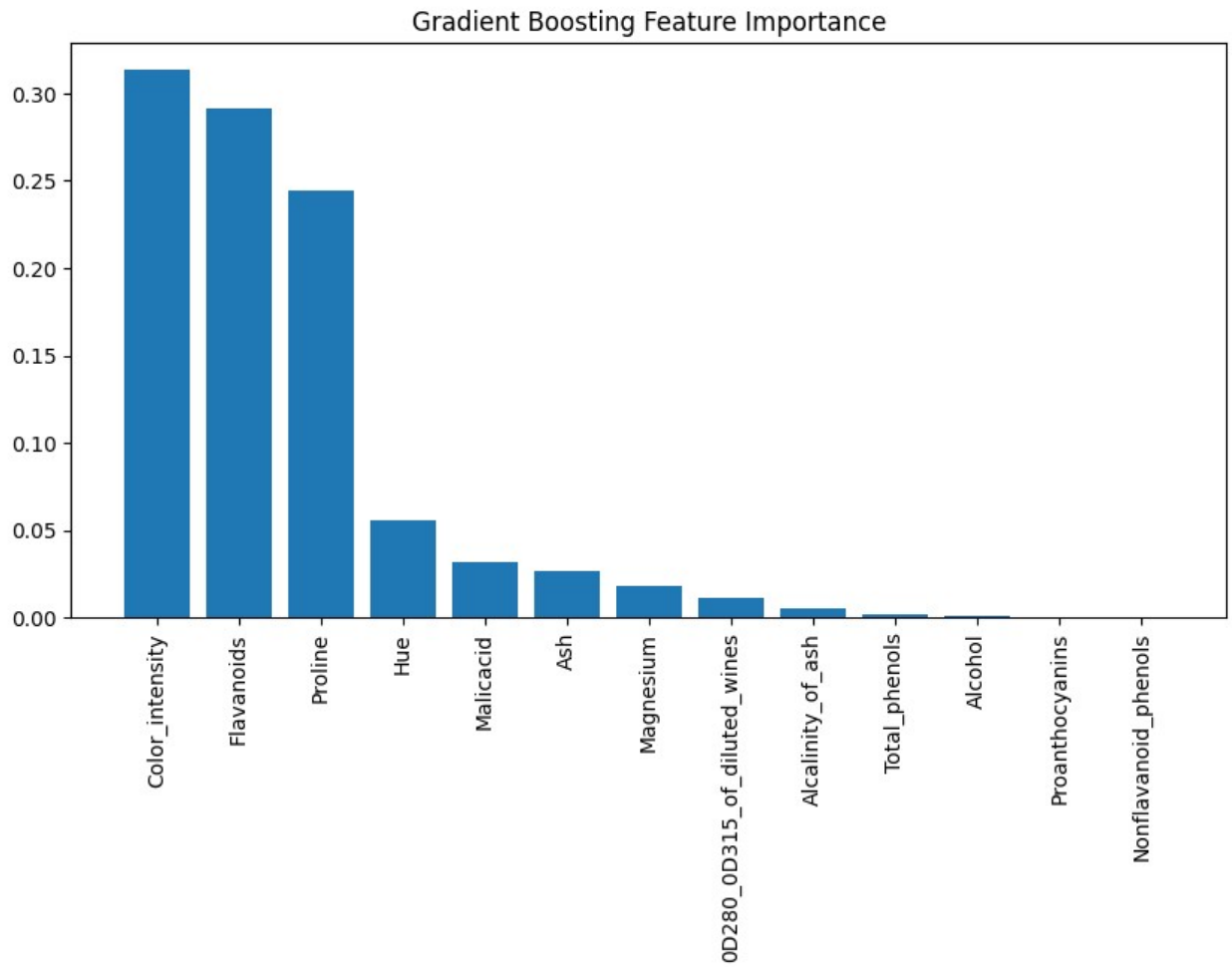



```
==================================================
AdaBoost
==================================================

Accuracy: 0.9166666666666666

Classification Report:

              precision    recall  f1-score   support

           0       0.86      1.00      0.92        12
           1       0.92      0.86      0.89        14
           2       1.00      0.90      0.95        10

    accuracy                           0.92        36
   macro avg       0.93      0.92      0.92        36
weighted avg       0.92      0.92      0.92        36
```

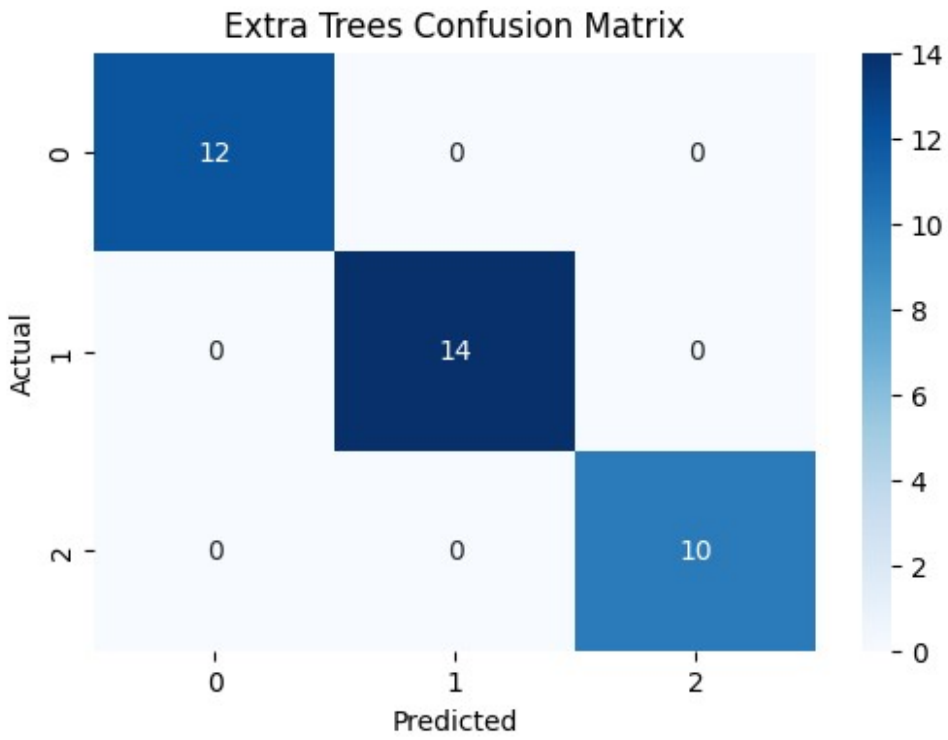


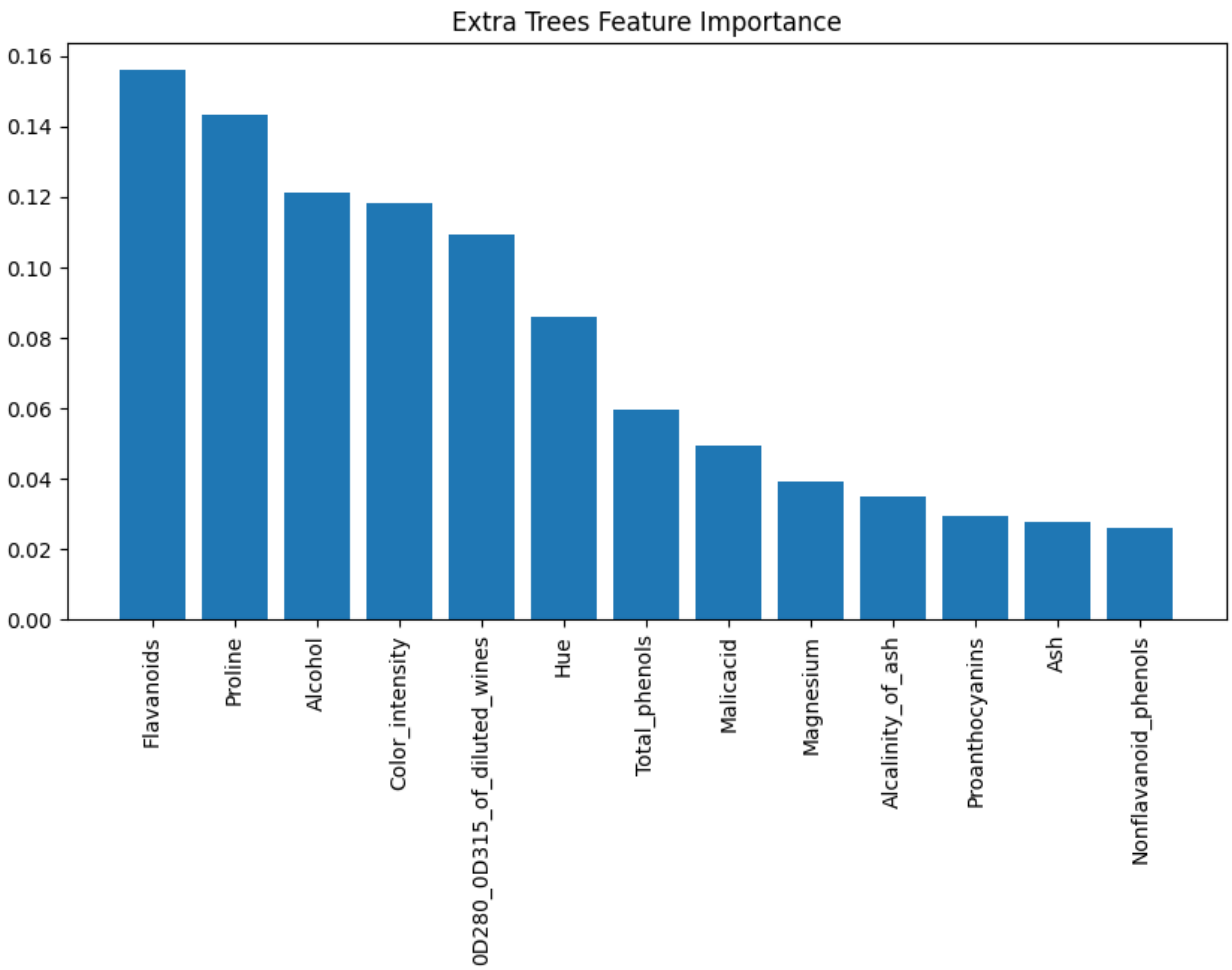

```
==================================================
Gradient Boosting
==================================================

Accuracy: 0.9444444444444444

Classification Report:

              precision    recall  f1-score   support

           0       0.92      1.00      0.96        12
           1       0.93      0.93      0.93        14
           2       1.00      0.90      0.95        10

    accuracy                           0.94        36
   macro avg       0.95      0.94      0.95        36
weighted avg       0.95      0.94      0.94        36
```


Gradient Boosting Confusion Matrix
Actual
Predicted
0
1
2
12
0
0
1
13
0
0
1
9
12
10
8
6
4
2
0

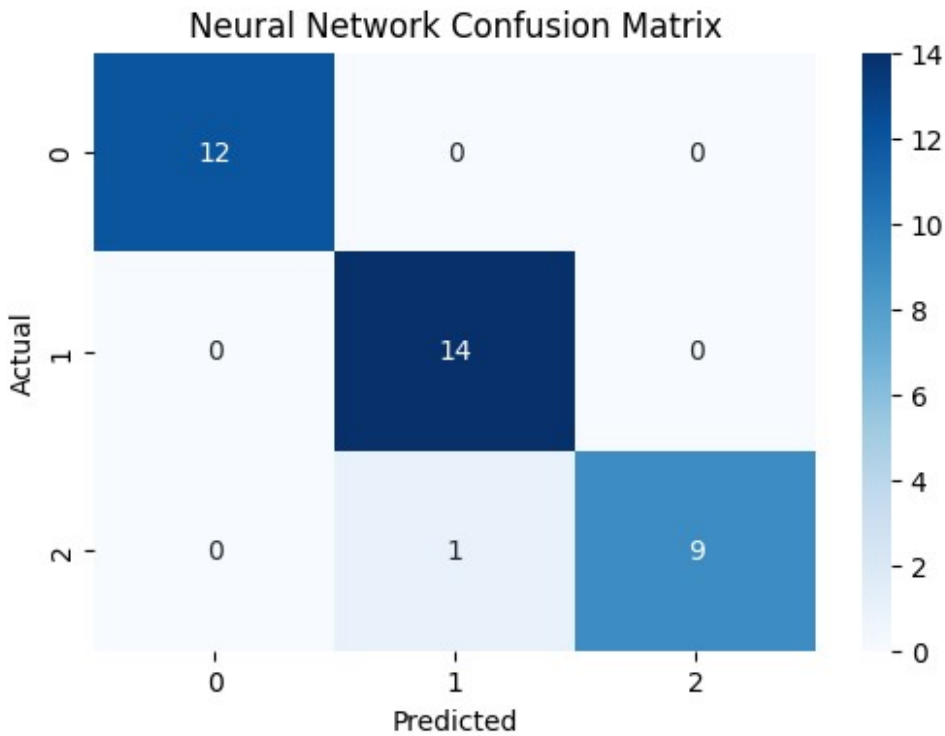


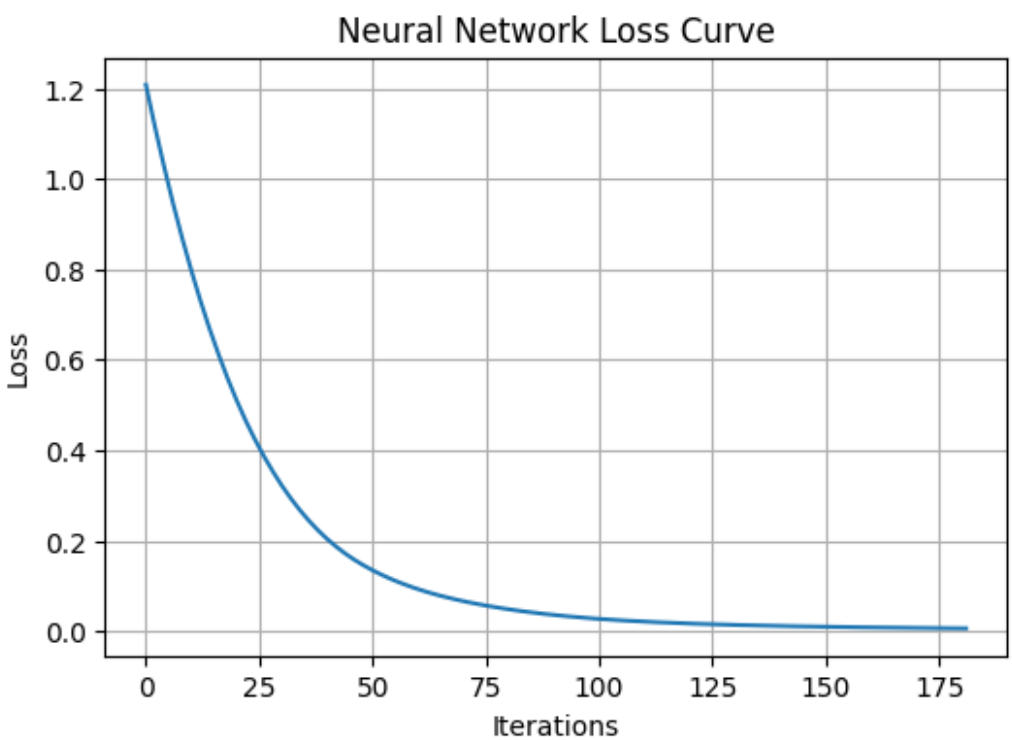

```
==================================================
Extra Trees
==================================================

Accuracy: 1.0

Classification Report:

              precision    recall  f1-score   support

           0       1.00      1.00      1.00        12
           1       1.00      1.00      1.00        14
           2       1.00      1.00      1.00        10

    accuracy                           1.00        36
   macro avg       1.00      1.00      1.00        36
weighted avg       1.00      1.00      1.00        36
```


Extra Trees Confusion Matrix
Actual
Predicted
0
1
2
12
0
0
0
14
0
0
0
10
14
12
10
8
6
4
2
0



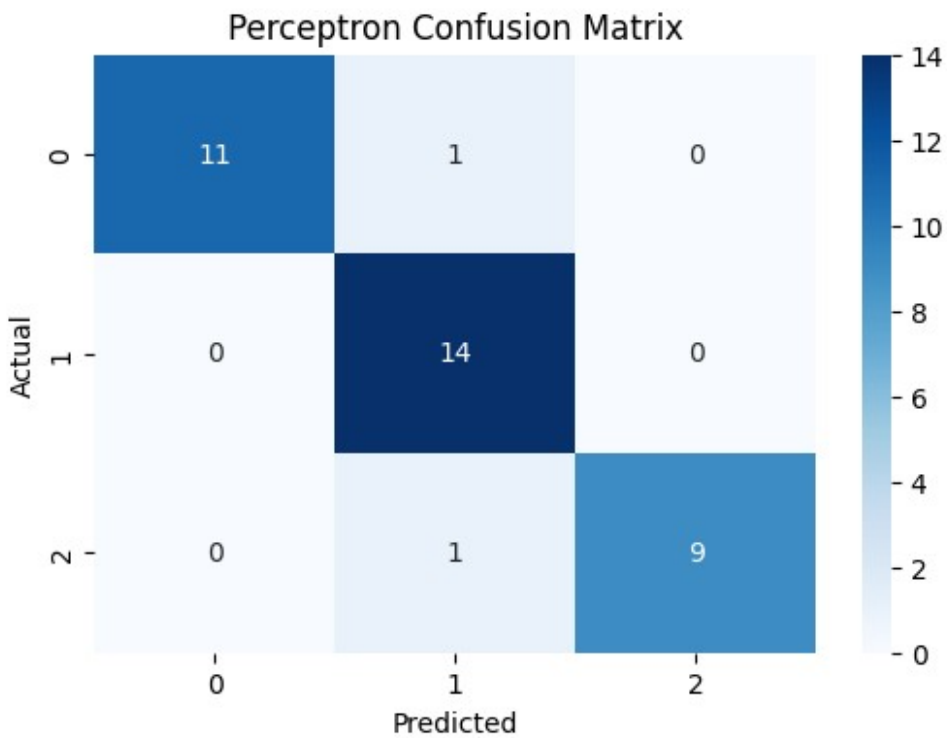

```
==================================================
Neural Network
==================================================

Accuracy: 0.9722222222222222

Classification Report:

              precision    recall  f1-score   support

           0       1.00      1.00      1.00        12
           1       0.93      1.00      0.97        14
           2       1.00      0.90      0.95        10

    accuracy                           0.97        36
   macro avg       0.98      0.97      0.97        36
weighted avg       0.97      0.97      0.97        36
```

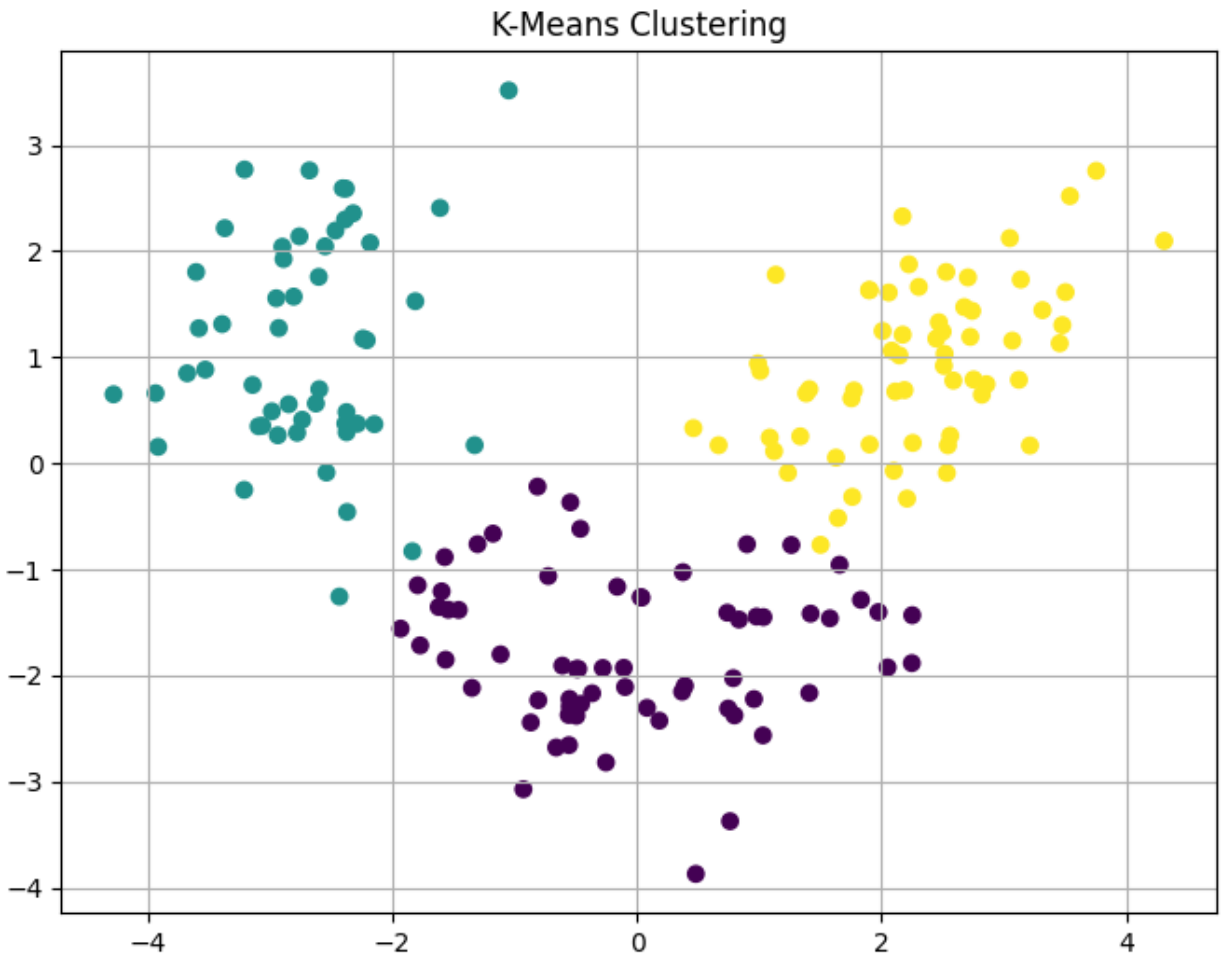

Neural Network Confusion Matrix
Actual
Predicted
0
1
2
12
0
0
0
14
0
0
1
9
14
12
10
8
6
4
2
0

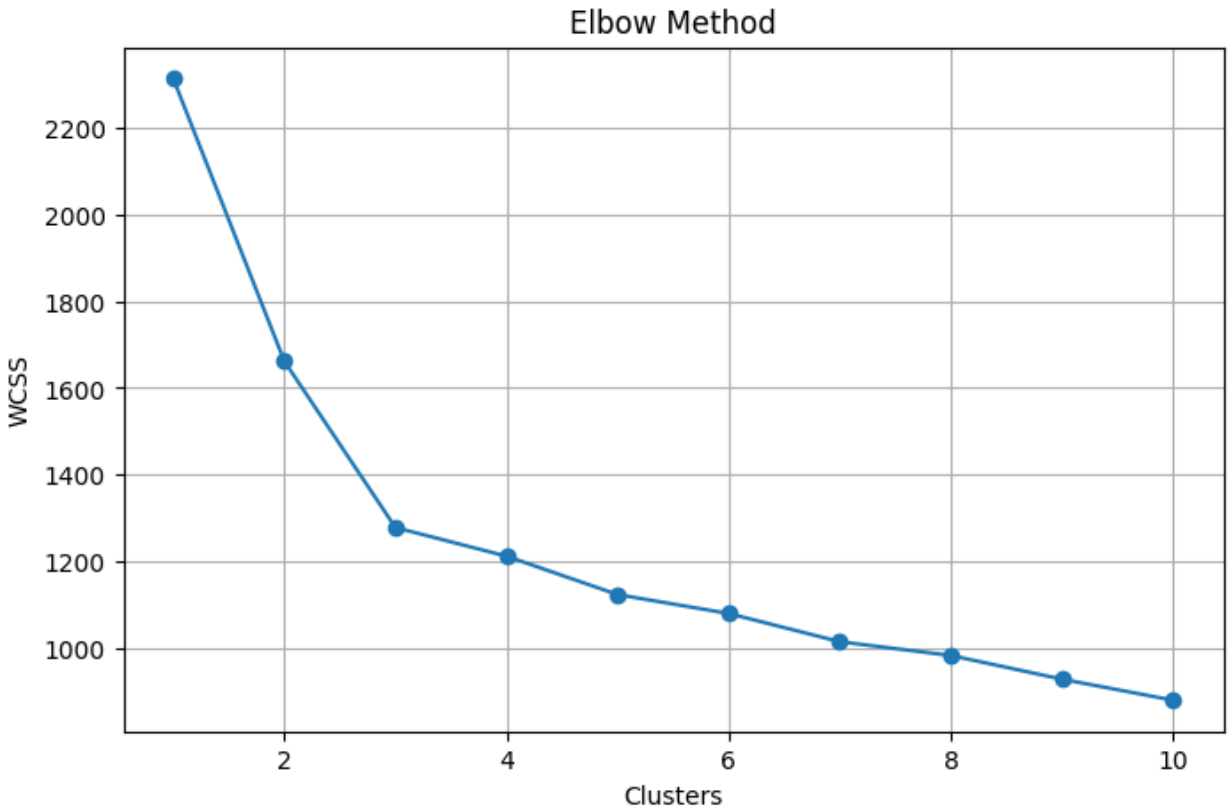

Neural Network Loss Curve
Loss
Iterations
1.2
1.0
0.8
0.6
0.4
0.2
0.0
0
25
50
75
100
125
150
175

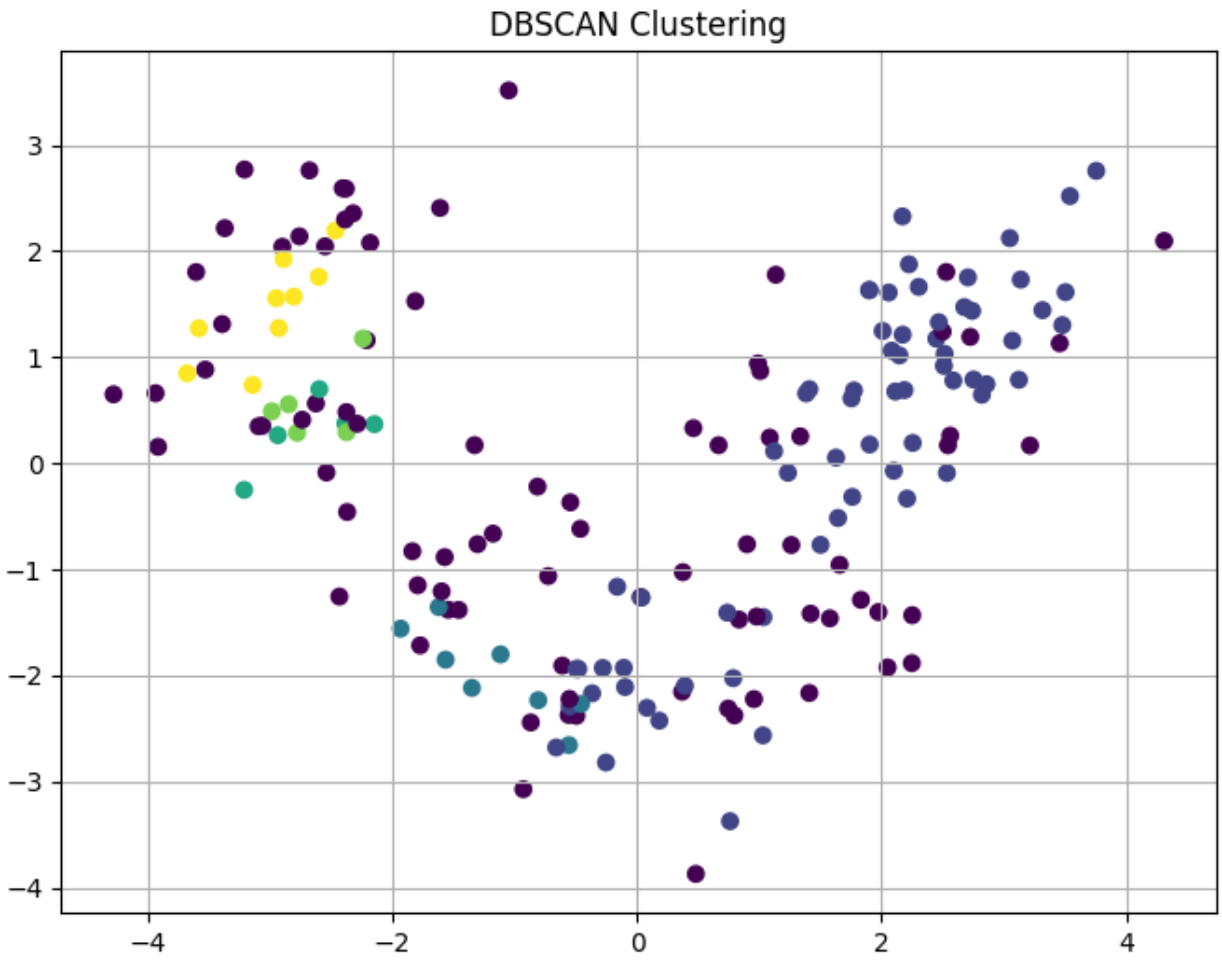
```
==============================================
Perceptron
```

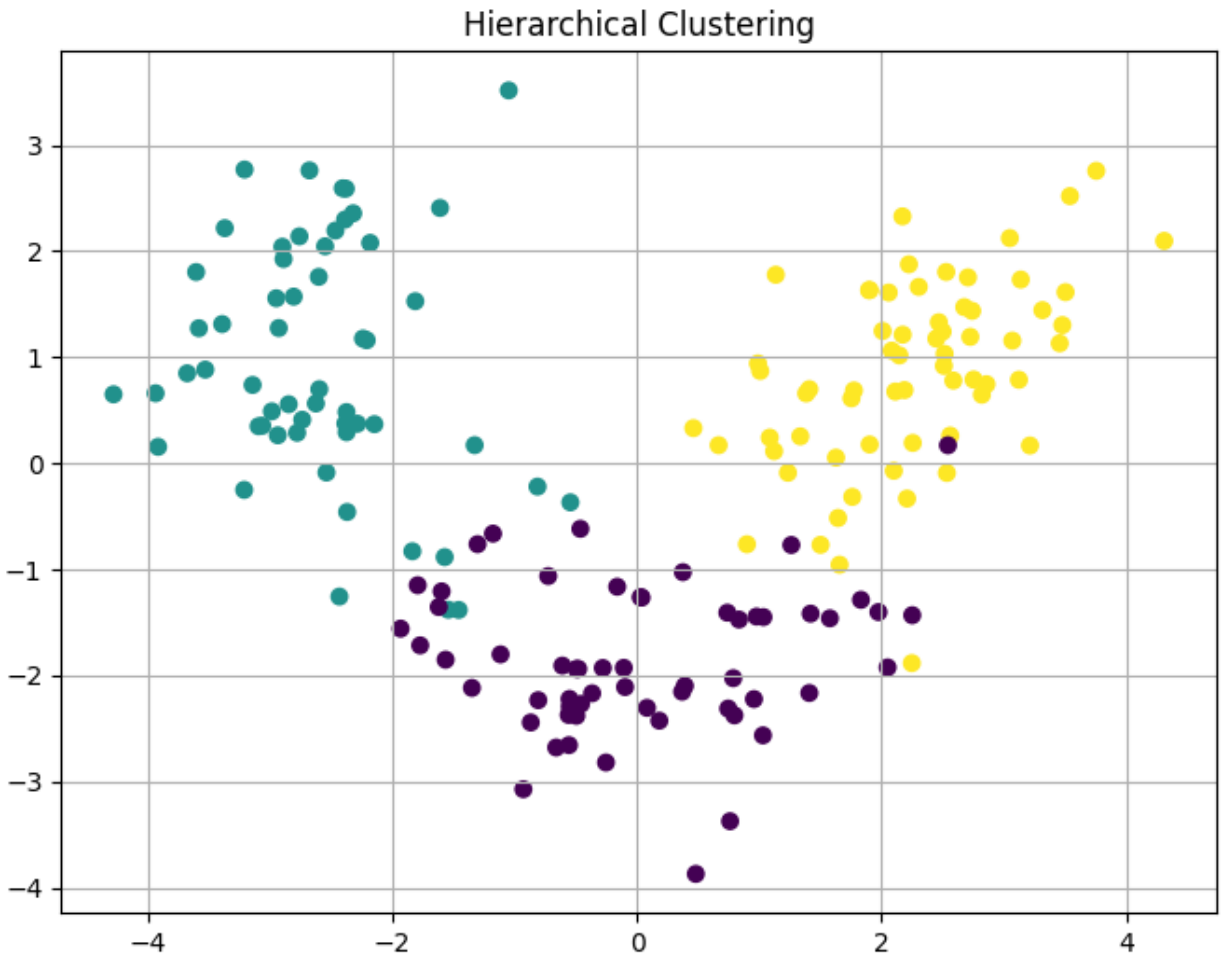
```
==================================================

Accuracy: 0.9444444444444444

Classification Report:

              precision    recall  f1-score   support

           0       1.00      0.92      0.96        12
           1       0.88      1.00      0.93        14
           2       1.00      0.90      0.95        10

    accuracy                           0.94        36
   macro avg       0.96      0.94      0.95        36
weighted avg       0.95      0.94      0.94        36
```

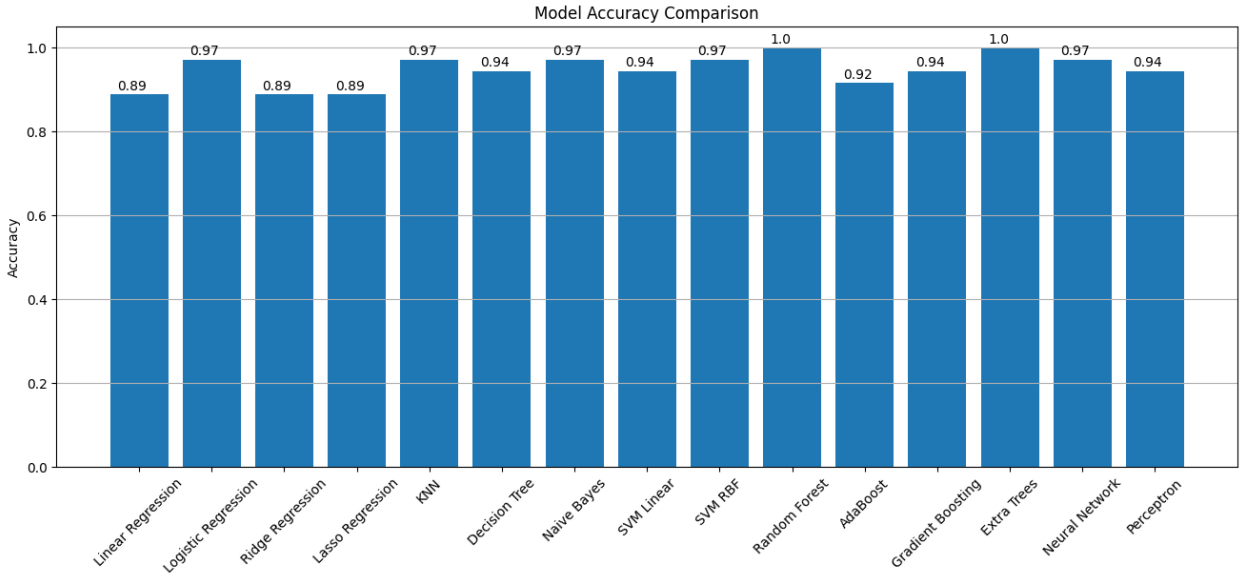


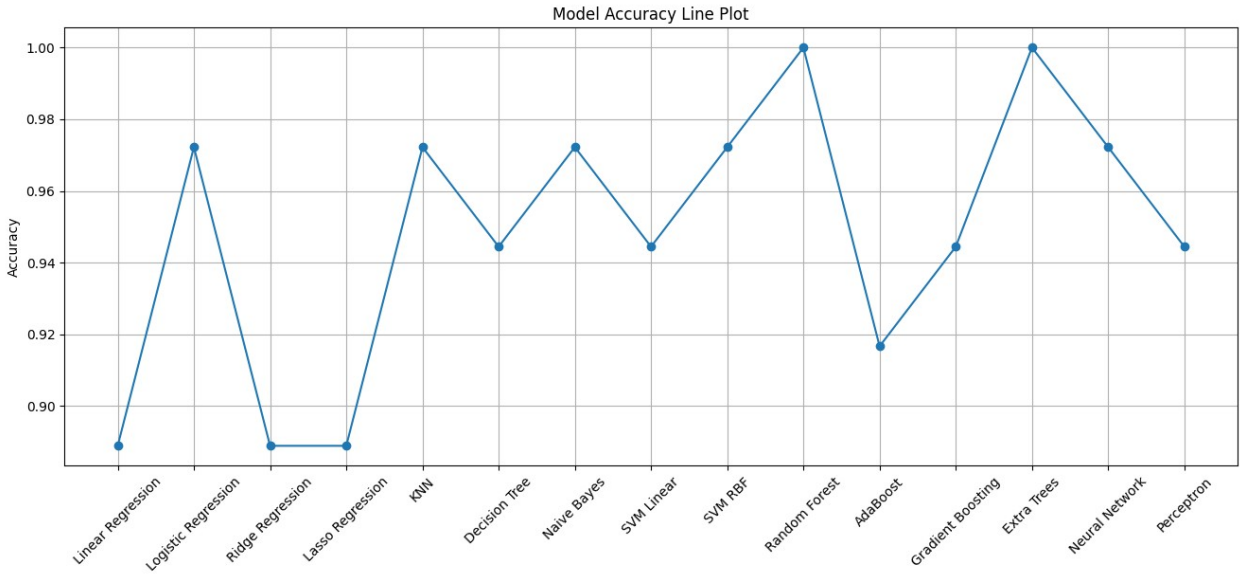

K-Means Clustering
3
2
1
0
−1
−2
−3
−4
−4
−2
0
2
4

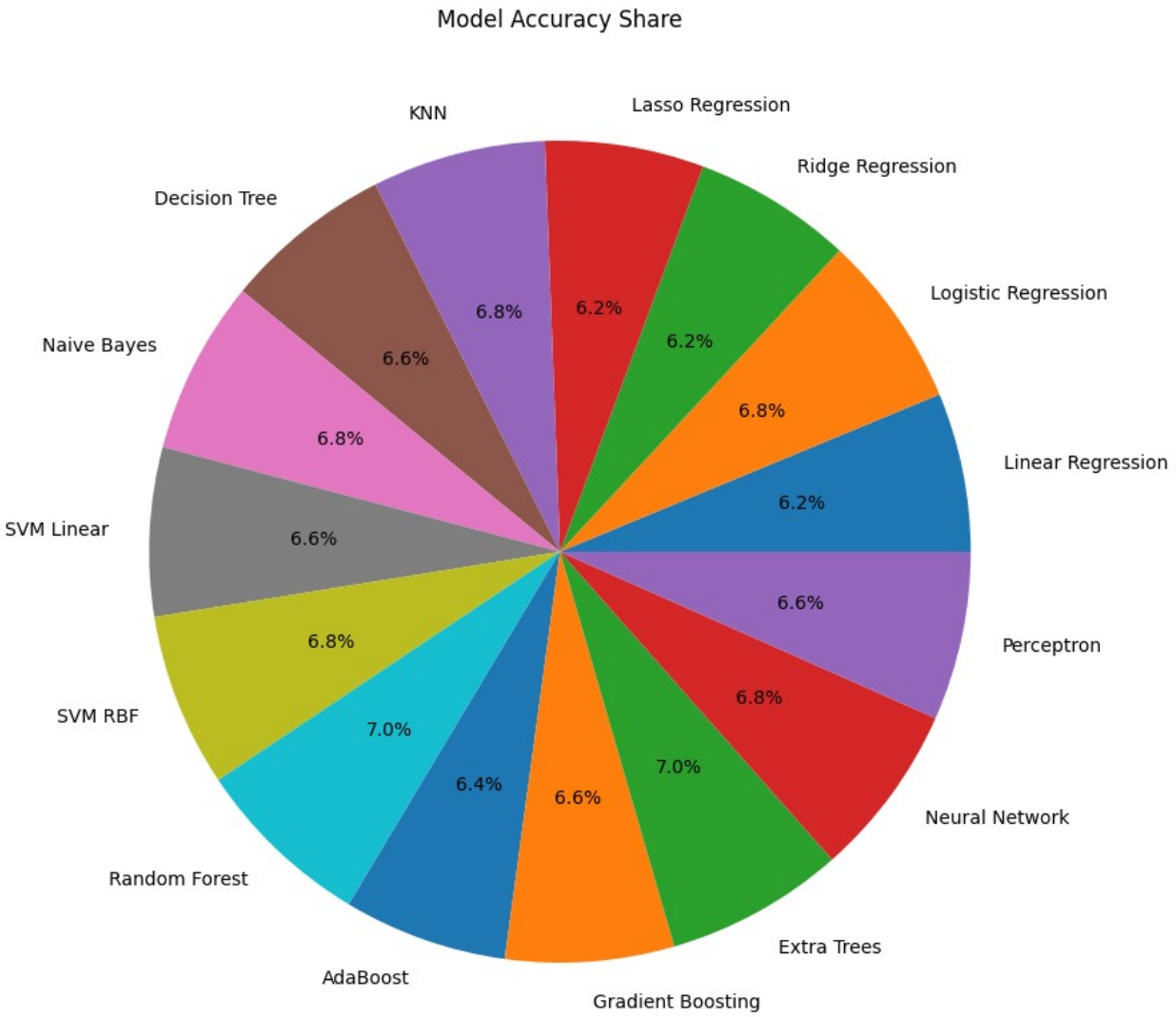

Elbow Method
WCSS
2200
2000
1800
1600
1400
1200
1000
2
4
6
8
10
Clusters

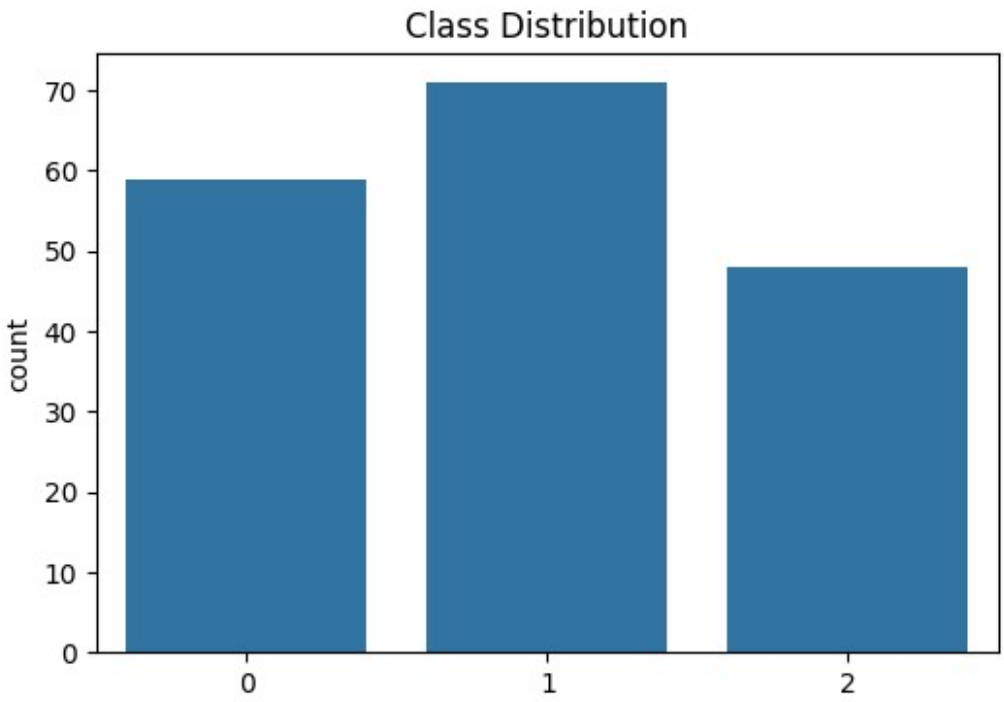

DBSCAN Clustering
3
2
1
0
−1
−2
−3
−4
−4
−2
0
2
4


Hierarchical Clustering
3
2
1
0
−1
−2
−3
−4
−4
−2
0
2
4


Model Accuracy Comparison
Accuracy
1.0
0.8
0.6
0.4
0.2
0.0
0.89
0.97
0.89
0.89
0.97
0.94
0.97
0.94
0.97
1.0
0.92
0.94
1.0
0.97
0.94
Linear Regression
Logistic Regression
Ridge Regression
Lasso Regression
KNN
Decision Tree
Naive Bayes
SVM Linear
SVM RBF
Random Forest
AdaBoost
Gradient Boosting
Extra Trees
Neural Network
Perceptron


Model Accuracy Line Plot
Accuracy
1.00
0.98
0.96
0.94
0.92
0.90
Linear Regression
Logistic Regression
Ridge Regression
Lasso Regression
KNN
Decision Tree
Naive Bayes
SVM Linear
SVM RBF
Random Forest
AdaBoost
Gradient Boosting
Extra Trees
Neural Network
Perceptron


Model Accuracy Share
KNN
Lasso Regression
Ridge Regression
Decision Tree
Logistic Regression
Naive Bayes
Linear Regression
SVM Linear
Perceptron
SVM RBF
Neural Network
Random Forest
Extra Trees
AdaBoost
Gradient Boosting
6.8%
6.2%
6.2%
6.6%
6.8%
6.2%
6.8%
6.6%
6.6%
6.8%
7.0%
6.4%
6.6%
7.0%




```
==================================================
BEST MODEL: Random Forest
BEST ACCURACY: 1.0
==================================================
```